DC_Complexlab Racoon POD 05 Purple
Headquarter (HQ) and CoomandPost (CP) should be implement

Implementation & Quick Reference Guide: POD 05 (Purple)

1. Central Quick Reference Data

Active Directory & Domain

- **FQDN Domain:** `purple.raccoon.lab`
- **NetBIOS Name:** `PURPLE`
- **AD Sites:** `HQ` (10.101.50.0/24) and `CP` (10.101.51.0/24)
- **DNS Conditional Forwarders:** `chq.raccoon.lab` → `10.100.101.254`, `wan.raccoon.lab` → `10.100.101.254`

Master Address Matrix

Site	Hostname	FQDN	Functio
HQ	`LR01`	`lr01.purple.raccoon.lab`	Router / WireGu
HQ	`HV01`	`hv01.purple.raccoon.lab`	Proxmo
HQ	`HQ-DC01`	`hq-dc01.purple.raccoon.lab`	Primary
HQ	`HQ-DC02`	`hq-dc02.purple.raccoon.lab`	Second DNS
HQ	`HQ-FS01`	`hq-fs01.purple.raccoon.lab`	File Ser

HQ	HQ-CA	hq-ca01.purple.raccoon.lab	Enterpr
HQ	HQ-SRV	hq-srv.purple.raccoon.lab	Linux (D Monitor
HQ	HQ-APC	hq-apc01.purple.raccoon.lab	Admin F
CP	LR02	lr02.purple.raccoon.lab	Router / WireGu
CP	HV02	hv02.purple.raccoon.lab	Proxmo
CP	CP-DC03	cp-dc03.purple.raccoon.lab	AD DS /
CP	CP-DC04	cp-dc04.purple.raccoon.lab	AD DS /
CP	CP-FS02	cp-fs02.purple.raccoon.lab	File Serv
CP	CP-APC	cp-apc01.purple.raccoon.lab	Admin F

WireGuard Tunnels

Tunnel Link	Subnet	First Endpoint	Second Endpc
CHQ-LR ↔ LR01	10.250.101.20/30	CHQ-LR : 10.250.101.21	LR01 : 10.250.101.
LR01 ↔ LR02	10.251.101.20/30	LR01 : 10.251.101.21	LR02 : 10.251.101.

2. Network Routing & Routing Tables

LR01 Routing Configuration

- **System Routing Table:**
  - `10.100.101.0/24` via `10.250.101.21` (dev `wg0`)
  - `10.101.51.0/24` via `10.251.101.22` (dev `wg1`)
  - `203.0.113.20/32` via `203.0.113.161` (dev `eth0`)
  - `0.0.0.0/0` via `203.0.113.161` (dev `eth0`)
- **NAT Rule:** SNAT/Masquerade outgoing traffic over `eth0` (WAN `203.0.113.162`) targeting `203.0.113.20` for package manager updates.

LR02 Routing Configuration

- **System Routing Table:**
  - `10.101.50.0/24` via `10.251.101.21` (dev `wg0`)
  - `10.100.101.0/24` via `10.251.101.21` (dev `wg0`)
  - `203.0.113.20/32` via `203.0.113.165` (dev `eth0`)
  - `0.0.0.0/0` via `203.0.113.165` (dev `eth0`)

WireGuard AllowedIPs Definitions

- **LR01 Peer CHQ-LR:** `10.250.101.21/32, 10.100.101.0/24`
- **LR01 Peer LR02:** `10.251.101.22/32, 10.101.51.0/24`
- **LR02 Peer LR01:** `10.251.101.21/32, 10.101.50.0/24, 10.100.101.0/24`

3. Implementation Plan

① APC Setup & Image Ingestion

Prerequisite for Proxmox guest deployment

1. Log into **HQ-APC** and **CP-APC** via RDP using local credentials (`.\remoteuser` / `Passw0rd`).

2. Open Windows File Explorer and navigate to `\\192.168.1.253\share` (User: `share`, Pass: `Passw0rd`).
3. Copy ISO installation images to the local `Downloads` folder on both APCs.
4. Access Proxmox web interface (`[https://10.101.50.10:8006](https://10.101.50.10:8006)` and `[https://10.101.51.10:8006](https://10.101.51.10:8006)`) using realm `Linux PAM` (`root` / `Passw0rd`).
5. Upload ISO files directly from APCs into local storage on **HV01** and **HV02**.

② Router Setup (LR01 & LR02)

Establish IP forwarding, WAN connectivity, and WireGuard tunnels

1. SSH into **LR01** (`10.101.50.254`) and **LR02** (`10.101.51.254`) using `linux_admin` / `Passw0rd`.
2. Enable IP forwarding (`sysctl -w net.ipv4.ip_forward=1`).
3. Configure WireGuard on **LR01**:
   - Set up tunnel `wg0` to CHQ-LR (`10.250.101.22/30` ↔ `10.250.101.21/30`).
   -

Set up tunnel wg1 to LR02 (10.251.101.21/30 ↔ 10.251.101.22/30).

4. Configure WireGuard on **LR02**:
   - Set up tunnel wg0 to LR01 (10.251.101.22/30 ↔ 10.251.101.21/30).
5. Add static routes on **LR01** and **LR02** for private cross-site routing.
6. Configure Source NAT on both LRs for WAN traffic destined for repository 203.0.113.20.
7. Verify cross-site connectivity: Ping 10.100.101.20 (CHQ www) and 10.101.51.254 (LR02) from **LR01**.

③ APC Routing Configuration

Prevent dual-gateway split-brain routing issues

1. Ensure default gateway is applied **ONLY** to the extNet interface on both APCs.
2. Open PowerShell as Administrator on **HQ-APC** and execute:

```
New-NetRoute -DestinationPrefix 10.101.51.0/24 -NextHop
10.101.50.254 -InterfaceAlias "LAN"
New-NetRoute -DestinationPrefix 10.100.101.0/24 -NextHop
10.101.50.254 -InterfaceAlias "LAN"
```

3. Open PowerShell as Administrator on **CP-APC** and execute:

```
New-NetRoute -DestinationPrefix 10.100.101.0/24 -NextHop
10.101.51.254 -InterfaceAlias "LAN"
New-NetRoute -DestinationPrefix 10.101.50.0/24 -NextHop
10.101.51.254 -InterfaceAlias "LAN"
```

④ Proxmox VM Provisioning

Deploy guest domain controllers and file servers

1. On **HV01**, create VMs for **HQ-DC01** (`10.101.50.11`), **HQ-DC02** (`10.101.50.12`), and **HQ-FS01** (`10.101.50.20`).
2. On **HV02**, create VMs for **CP-DC03** (`10.101.51.11`), **CP-DC04** (`10.101.51.12`), and **CP-FS02** (`10.101.51.20`).
3. Install Windows Server OS on all 6 VMs. Assign static IPs, network masks (`255.255.255.0`), and local gateways (`.254`).

⑤ Active Directory & DNS Deployment

Build domain forest purple.raccoon.lab across both sites

1. Provision **HQ-DC01**: Promote to Active Directory Domain Services (AD DS) creating new forest `purple.raccoon.lab`. Set DNS server target to self (`127.0.0.1`).
2. Promote **HQ-DC02** as replica DC in `purple.raccoon.lab`. Set Primary DNS to `10.101.50.11`, Secondary to `127.0.0.1`.
3. Configure **AD Sites and Services**:

- Rename `Default-First-Site-Name` to HQ. Associate subnet `10.101.50.0/24`.
- Create new site CP. Associate subnet `10.101.51.0/24`.

4. Promote **CP-DC03** and **CP-DC04** into site CP.
5. Set DNS server network properties on all domain hosts to point to local site DCs first, remote site DCs second.
6. Add Conditional Forwarders in DNS on all DCs:
   - `chq.raccoon.lab` → `10.100.101.254`
   - `wan.raccoon.lab` → `10.100.101.254`
7. Join **HQ-FS01**, **CP-FS02**, **HQ-APC**, and **CP-APC** to domain `purple.raccoon.lab`. Ensure APCs register *only* their LAN IPs in DNS.

⑥ PKI & Certificate Infrastructure

Integrate Enterprise SubCA with Classroom Root CA

1. On **HQ-CA** (`10.101.50.30`), install Active Directory Certificate Services (AD CS).
2.

Configure AIA and CDP distribution point URLs pointing to `[http://pki.chq.raccoon.lab/](http://pki.chq.raccoon.lab/)` (`10.100.101.20`).

3.

Generate a Certificate Signing Request (CSR) for an Enterprise Subordinate CA.

4.

Submit CSR to the trainer to be signed by Classroom Root CA (`Root-CA` / `chq-ca01`).

5.

Import the signed certificate and complete AD CS configuration on **HQ-CA**.

6.

Publish Root CA and SubCA certificates and CRLs into Active Directory via GPO/PKI publishing so all HQ/CP nodes trust the chain.

⑦ HQ-SRV Linux Infrastructure

Prepare base host for Docker services

1.

Configure network interfaces on **HQ-SRV** (`10.101.50.40/24`, Gateway: `10.101.50.254`).

2.

Configure APT/package manager sources to use `[http://repository.wan.raccoon.lab](http://repository.wan.raccoon.lab)` (`203.0.113.20`).

3.

Install Docker engine and dependencies.

4.

Note for subsequent tasks: Ensure custom Docker bridges use subnets that do

not collide with `10.100.0.0/16`, `10.101.0.0/16`, `10.250.0.0/16`, `10.251.0.0/16`, or `203.0.113.0/24`.

4. Verification Commands

Run these standard verification commands from the respective nodes once configuration steps are completed:

Network & Routing Tests

Bash

```
# Executed from LR01: Verify WireGuard status and routes
wg show
ip route show

# Ping across WireGuard links from HQ to CP and CentralHQ
ping -c 2 10.251.101.22
ping -c 2 10.101.51.254
ping -c 2 10.100.101.20
```

Windows Route Verification

PowerShell

```
# Executed on HQ-APC or CP-APC: Confirm routing table and binding
Get-NetRoute -AddressFamily IPv4 | Where-Object {$_.NextHop -ne "0.0.0.0"}
Test-NetConnection -ComputerName 10.101.51.11 -Port 135
```

Active Directory & DNS Validation

PowerShell

```
# Executed on HQ-DC01 or CP-DC03
dcdiag /v
repadmin /showrepl
Resolve-DnsName www.chq.raccoon.lab
```

PKI Validation

DOS

```
:: Executed on domain-joined Windows node
certutil -ping pki.chq.raccoon.lab
certutil -VerifyStore Root
```

how to get all interfaces on LR01 && LR02

and

Here are the precise commands to check network interfaces and configure the WireGuard tunnels on **LR01** and **LR02** for **POD 05 (Purple)**.

1. Retrieve All Interfaces on LR01 & LR02

SSH into **LR01** (`10.101.50.254`) or **LR02** (`10.101.51.254`) with user `linux_admin` / `Passw0rd` and run:

Bash

```
# List all interfaces with IP addresses and link state
```

```
# List all interfaces with IP addresses and link state
ip -br addr show

# Alternative detailed interface status
ip link show
```

Identify your interface names: Note which physical interface corresponds to the **LAN** (`eth0` / `ens18` style) and **WAN** (`eth1` / `ens19` style) before generating WireGuard keys.

2. Configure WireGuard on LR01

Log in to **LR01** as `root` (or run with `sudo`).

Step 1: Generate WireGuard Keys

Bash

```
umask 077
wg genkey | tee /etc/wireguard/lr01_private.key | wg pubkey > /etc/wireguard/lr0
```

Step 2: Create Tunnel `/etc/wireguard/wg0.conf` (Link to CHQ-LR)

Ini, TOML

```
[Interface]
PrivateKey = <CONTENTS_OF_lr01_private.key>
Address = 10.250.101.22/30
ListenPort = 51820

[Peer]
# CHQ-LR Public Key (Provided by trainer or CHQ environment)
PublicKey = <CHQ_LR_PUBLIC_KEY>
Endpoint = 203.0.113.10:51820
AllowedIPs = 10.250.101.21/32, 10.100.101.0/24
PersistentKeepalive = 25
```

Step 3: Create Tunnel `/etc/wireguard/wg1.conf` (Link to LR02)

Ini, TOML

```
[Interface]
PrivateKey = <CONTENTS_OF_lr01_private.key>
Address = 10.251.101.21/30
ListenPort = 51821

[Peer]
# LR02 Public Key (Generated on LR02 in next step)
PublicKey = <LR02_PUBLIC_KEY>
Endpoint = 203.0.113.166:51820
AllowedIPs = 10.251.101.22/32, 10.101.51.0/24
PersistentKeepalive = 25
```

Step 4: Enable Tunnels and Persistent Routes

Bash

```
# Enable IPv4 Forwarding
sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf

# Start interfaces and enable at boot
systemctl enable --now wg-quick@wg0
systemctl enable --now wg-quick@wg1

# Add static routes for cross-site reaching
ip route add 10.100.101.0/24 dev wg0
ip route add 10.101.51.0/24 dev wg1
```

3. Configure WireGuard on LR02

Log in to **LR02** as `root` (or run with `sudo`).

Step 1: Generate WireGuard Keys

Bash

```
umask 077
wg genkey | tee /etc/wireguard/lr02_private.key | wg pubkey > /etc/wireguard/lr0
```

Step 2: Create Tunnel `/etc/wireguard/wg0.conf` (Link to LR01)

Ini, TOML

```
[Interface]
PrivateKey = <CONTENTS_OF_lr02_private.key>
Address = 10.251.101.22/30
ListenPort = 51820

[Peer]
# LR01 Public Key (From /etc/wireguard/lr01_public.key)
PublicKey = <LR01_PUBLIC_KEY>
Endpoint = 203.0.113.162:51821
AllowedIPs = 10.251.101.21/32, 10.101.50.0/24, 10.100.101.0/24
PersistentKeepalive = 25
```

Step 3: Enable Tunnel and Persistent Routes

Bash

```
# Enable IPv4 Forwarding
sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf

# Start interface and enable at boot
systemctl enable --now wg-quick@wg0
```

```
# Add static routes (LR02 routes both HQ LAN and CHQ LAN via LR01)
ip route add 10.101.50.0/24 dev wg0
ip route add 10.100.101.0/24 dev wg0
```

4. Verification

After exchanging public keys and bringing up interfaces on both routers:

Bash

```
# Check status of interfaces and peers on LR01 / LR02
wg show

# Test IP reachability across the tunnels from LR02
ping -c 2 10.251.101.21   # Ping LR01 WireGuard IP
ping -c 2 10.101.50.254    # Ping LR01 HQ LAN IP
ping -c 2 10.100.101.20    # Ping CHQ Web Server via LR01 transit
```

lr01 && lr02 it is Ubuntu + nftables

Here is the complete setup for **LR01** and **LR02** on **Ubuntu** using `wg-quick` and `nftables`.

1. View All Interfaces

Run this on both routers to inspect network cards and IP assignments:

Bash

```
# Quick summary of interfaces and assigned IPs
ip -br a

# View active nftables ruleset
sudo nft list ruleset
```

2. Configure LR01 (HQ Router)

Step 1: Enable IP Forwarding

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.d/99-ipforward.conf
sudo sysctl -p /etc/sysctl.d/99-ipforward.conf
```

Step 2: Generate Keys

Bash

```
sudo mkdir -p /etc/wireguard && cd /etc/wireguard
umask 077
sudo wg genkey | sudo tee lr01_private.key | wg pubkey | sudo tee lr01_public.ke
```

Step 3: Create WireGuard Interfaces

`/etc/wireguard/wg0.conf` (CHQ Link):

Ini, TOML

```
[Interface]
PrivateKey = <CONTENTS_OF_lr01_private.key>
```

```
Address = 10.250.101.22/30
ListenPort = 51820

[Peer]
# CHQ-LR Public Key (Provided by Trainer)
PublicKey = <CHQ_LR_PUBLIC_KEY>
Endpoint = 203.0.113.10:51820
AllowedIPs = 10.250.101.21/32, 10.100.101.0/24
PersistentKeepalive = 25
```

`/etc/wireguard/wg1.conf` (CP Link):

Ini, TOML

```
[Interface]
PrivateKey = <CONTENTS_OF_lr01_private.key>
Address = 10.251.101.21/30
ListenPort = 51821

[Peer]
# LR02 Public Key (From LR02 Step 2)
PublicKey = <LR02_PUBLIC_KEY>
Endpoint = 203.0.113.166:51820
AllowedIPs = 10.251.101.22/32, 10.101.51.0/24
PersistentKeepalive = 25
```

Step 4: Configure `nftables` Firewall (`/etc/nftables.conf`)

Replace `<WAN_IF>` with your WAN interface name (e.g., `eth0` or `ens19`):

Code snippet

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {
```

```
    chain input {
        type filter hook input priority filter; policy drop;

        iif "lo" accept
        ct state established,related accept

        # Allow SSH and ICMP
        tcp dport 22 accept
        icmp type echo-request accept

        # Allow WireGuard inbound on WAN
        udp dport { 51820, 51821 } accept
    }

    chain forward {
        type filter hook forward priority filter; policy drop;

        ct state established,related accept

        # Allow cross-site transit between HQ, CP, and CHQ
        iifname "wg0" oifname "wg1" accept
        iifname "wg1" oifname "wg0" accept

        # Allow LAN traffic to/from VPN tunnels
        ip saddr 10.101.50.0/24 oifname { "wg0", "wg1" } accept
        iifname { "wg0", "wg1" } ip daddr 10.101.50.0/24 accept

        # Allow LAN/Router access to WAN Repository (203.0.113.20)
        ip daddr 203.0.113.20 accept
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}

table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;

        # Source NAT for Linux Repository updates over WAN
        ip daddr 203.0.113.20 oif "<WAN_IF>" masquerade
    }
}
```

Step 5: Start Services

Bash

```
sudo systemctl enable --now nftables
sudo systemctl enable --now wg-quick@wg0
sudo systemctl enable --now wg-quick@wg1
```

3. Configure LR02 (CP Router)

Step 1: Enable IP Forwarding

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.d/99-ipforward.conf
sudo sysctl -p /etc/sysctl.d/99-ipforward.conf
```

Step 2: Generate Keys

Bash

```
sudo mkdir -p /etc/wireguard && cd /etc/wireguard
umask 077
sudo wg genkey | sudo tee lr02_private.key | wg pubkey | sudo tee lr02_public.ke
```

Step 3: Create WireGuard Interface

`/etc/wireguard/wg0.conf` (HQ Link):

Ini, TOML

```ini
[Interface]
PrivateKey = <CONTENTS_OF_lr02_private.key>
Address = 10.251.101.22/30
ListenPort = 51820

[Peer]
# LR01 Public Key (From LR01 Step 2)
PublicKey = <LR01_PUBLIC_KEY>
Endpoint = 203.0.113.162:51821
AllowedIPs = 10.251.101.21/32, 10.101.50.0/24, 10.100.101.0/24
PersistentKeepalive = 25
```

Step 4: Configure `nftables` Firewall (`/etc/nftables.conf`)

Replace `<WAN_IF>` with your WAN interface name:

Code snippet

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;

        iif "lo" accept
        ct state established,related accept

        # Allow SSH and ICMP
        tcp dport 22 accept
        icmp type echo-request accept

        # Allow WireGuard inbound on WAN
        udp dport 51820 accept
    }

    chain forward {
        type filter hook forward priority filter; policy drop;
```

```
        ct state established,related accept

        # Allow LAN traffic to/from VPN tunnel (HQ & CHQ via LR01)
        ip saddr 10.101.51.0/24 oifname "wg0" accept
        iifname "wg0" ip daddr 10.101.51.0/24 accept

        # Allow LAN/Router access to WAN Repository (203.0.113.20)
        ip daddr 203.0.113.20 accept
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}

table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;

        # Source NAT for Linux Repository updates over WAN
        ip daddr 203.0.113.20 oif "<WAN_IF>" masquerade
    }
}
```

Step 5: Start Services

Bash

```
sudo systemctl enable --now nftables
sudo systemctl enable --now wg-quick@wg0
```

4. Connectivity Verification

Execute from **LR02**:

Bash

Bash

```
# Check WireGuard handshake status
sudo wg show

# Test WireGuard tunnel link to LR01
ping -c 2 10.251.101.21

# Test reachability to HQ LAN Gateway
ping -c 2 10.101.50.254

# Test reachability to Central CHQ Web Server via LR01 transit
ping -c 2 10.100.101.20
```

configure hq-src with static ip and enabled to update apt via 203.0.113.20

To configure **HQ-SRV** with a static IP and point its APT package manager to the classroom repository at `203.0.113.20`, follow these steps on **HQ-SRV**.

1. Configure Static IP (`netplan`)

Ubuntu uses Netplan for network configuration. Edit your Netplan config file (typically `/etc/netplan/01-netcfg.yaml` or `/etc/netplan/50-cloud-init.yaml`).

Bash

```
sudo nano /etc/netplan/01-netcfg.yaml
```

Replace `<INTERFACE_NAME>` with your actual interface (e.g., `eth0` or `ens18`, check using `ip -br a`):

YAML

```yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    <INTERFACE_NAME>:
      dhcp4: no
      addresses:
        - 10.101.50.40/24
      routes:
        - to: default
          via: 10.101.50.254
      nameservers:
        addresses:
          - 10.101.50.11
          - 10.101.50.12
```

Apply the network configuration:

Bash

```bash
sudo netplan apply
```

2. Configure APT to Use Local Repository (203.0.113.20)

Backup your existing `sources.list` file and create a new one pointing to `repository.wan.raccoon.lab / 203.0.113.20` .

Bash

```bash
# Backup existing sources
sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak
```

```
# Update /etc/hosts to ensure repository resolution works before DNS is live
echo "203.0.113.20 repository.wan.raccoon.lab" | sudo tee -a /etc/hosts
```

Edit `/etc/apt/sources.list` :

Bash

```
sudo nano /etc/apt/sources.list
```

Add the repository configuration pointing to `203.0.113.20` (or `repository.wan.raccoon.lab`):

Code snippet

```
deb http://203.0.113.20/ubuntu noble main restricted universe multiverse
deb http://203.0.113.20/ubuntu noble-updates main restricted universe multiverse
deb http://203.0.113.20/ubuntu noble-security main restricted universe multivers
```

(Note: Replace `noble` *with your Ubuntu release codename if using another version, e.g.,* `jammy` *for 22.04).*

3. Verify Configuration

Test network reachability and update the APT package cache:

Bash

```
# Verify gateway connectivity
ping -c 2 10.101.50.254

# Verify repository connectivity via LR01 SNAT
ping -c 2 203.0.113.20
```

```
# Test APT update from local repository
sudo apt update
```

configure DEBIAN hv01 with static

To configure **HV01** (Debian / Proxmox VE host) with a static IP address, edit the Debian `/etc/network/interfaces` file.

Proxmox VE uses a Linux Network Bridge (`vmbr0`) tied to your physical interface.

1. Identify Physical Interface Name

Check the active physical network card name:

Bash

```
ip -br link
```

(Note down your physical interface name, e.g., `eno1` , `eth0` , or `enp3s0`).

2. Configure `/etc/network/interfaces`

Edit the network configuration file:

Bash

```
nano /etc/network/interfaces
```

Replace `<PHYSICAL_IF>` with your actual interface name (e.g., `eth0`), and ensure the configuration matches **POD 05 (Purple)** specifications:

Code snippet

```
auto lo
iface lo inet loopback

iface <PHYSICAL_IF> inet manual

auto vmbr0
iface vmbr0 inet static
        address 10.101.50.10/24
        gateway 10.101.50.254
        bridge-ports <PHYSICAL_IF>
        bridge-stp off
        bridge-fd 0
```

3. Configure Local Host Name & DNS Resolution

Ensure `/etc/hosts` matches your static IP and FQDN:

Bash

```
nano /etc/hosts
```

Set the contents to:

Plaintext

```
127.0.0.1 localhost
10.101.50.10 hv01.purple.raccoon.lab hv01

# The following lines are desirable for IPv6 capable hosts
```

```
# The following lines are desirable for IPv6 capable hosts
::1     localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Set DNS nameservers in `/etc/resolv.conf`:

Bash

```
nano /etc/resolv.conf
```

Plaintext

```
search purple.raccoon.lab
nameserver 10.101.50.11
nameserver 10.101.50.12
```

4. Apply Network Settings

Restart networking services to apply the static IP assignment:

Bash

```
ifreload -a
# Or apply by restarting the networking service
systemctl restart networking
```

5. Verification

Verify the static IP assignment and gateway route:

Bash

```
# Verify IP is set on vmbr0
ip -br addr show vmbr0

# Test reachability to LR01 gateway
ping -c 2 10.101.50.254

# Test reachability to repository via LR01 SNAT
ping -c 2 203.0.113.20
```

how to add debian apt repo on 203.0.113.20

To configure Debian on **HV01** to use the central package repository at `203.0.113.20`, update your APT sources file.

1. Set Temporary Host Override

Add the repository hostname mapping to `/etc/hosts` so APT resolution works immediately:

Bash

```
echo "203.0.113.20 repository.wan.raccoon.lab" | sudo tee -a /etc/hosts
```

2. Configure `/etc/apt/sources.list`

Backup your original configuration and replace it with the repository settings:

Bash

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak
```

Edit `/etc/apt/sources.list` :

Bash

```
sudo nano /etc/apt/sources.list
```

For Debian 12 (Bookworm)

Code snippet

```
deb http://203.0.113.20/debian bookworm main contrib non-free non-free-firmware
deb http://203.0.113.20/debian bookworm-updates main contrib non-free non-free-f
deb http://203.0.113.20/debian-security bookworm-security main contrib non-free
```

For Debian 11 (Bullseye)

Code snippet

```
deb http://203.0.113.20/debian bullseye main contrib non-free
deb http://203.0.113.20/debian bullseye-updates main contrib non-free
deb http://203.0.113.20/debian-security bullseye-security main contrib non-free
```

3. Proxmox VE No-Subscription Repository (Optional)

If **HV01** runs Proxmox VE on top of Debian, disable the enterprise commercial repo and add the pve-no-subscription entry pointing to `203.0.113.20`:

Bash

```
# Disable enterprise repository list
sudo comment-out or remove entries in /etc/apt/sources.list.d/pve-enterprise.lis

# Create no-subscription list
echo "deb http://203.0.113.20/proxmox/wiki pve-no-subscription" | sudo tee /etc/
```

4. Apply and Update Cache

Verify route reachability through **LR01** and fetch package metadata:

Bash

```
# Verify network path to repository
ping -c 2 203.0.113.20

# Update APT repository cache
sudo apt update
```

how to install && configure WireGuard on LR01?

To install and configure WireGuard on **LR01** (Ubuntu + nftables), follow these steps.

As defined in the **POD 05 (Purple)** plan, **LR01** acts as the central hub router connecting to **CHQ-LR** via interface `wg0` and to **LR02 (CP)** via interface `wg1`.

Step 1: Install WireGuard

Log in to **LR01** as `root` (or use `sudo`) and run:

Bash

```
sudo apt update
sudo apt install -y wireguard wireguard-tools
```

Step 2: Generate Key Pairs

Generate private and public keys for **LR01**:

Bash

```
sudo mkdir -p /etc/wireguard
cd /etc/wireguard
umask 077
sudo wg genkey | sudo tee lr01_private.key | wg pubkey | sudo tee lr01_public.ke
```

Note: Run `cat /etc/wireguard/lr01_public.key` to view LR01's public key. Share this key with the trainer (for **CHQ-LR**) and set it up on **LR02**.

Step 3: Create WireGuard Interfaces

1. Interface `wg0` (Link to CHQ-LR)

Create `/etc/wireguard/wg0.conf`:

Bash

```
sudo nano /etc/wireguard/wg0.conf
```

Paste the following configuration:

Ini, TOML

```
[Interface]
PrivateKey = <INSERT_CONTENTS_OF_lr01_private.key>
Address = 10.250.101.22/30
ListenPort = 51820

[Peer]
# CHQ-LR Public Key (Provided by Trainer)
PublicKey = <INSERT_CHQ_LR_PUBLIC_KEY>
Endpoint = 203.0.113.10:51820
AllowedIPs = 10.250.101.21/32, 10.100.101.0/24
PersistentKeepalive = 25
```

2. Interface `wg1` (Link to LR02)

Create `/etc/wireguard/wg1.conf`:

Bash

```
sudo nano /etc/wireguard/wg1.conf
```

Paste the following configuration:

Ini, TOML

```
[Interface]
PrivateKey = <INSERT_CONTENTS_OF_lr01_private.key>
Address = 10.251.101.21/30
ListenPort = 51821
```

```
[Peer]
# LR02 Public Key (Generated on LR02)
PublicKey = <INSERT_LR02_PUBLIC_KEY>
Endpoint = 203.0.113.166:51820
AllowedIPs = 10.251.101.22/32, 10.101.51.0/24
PersistentKeepalive = 25
```

Step 4: Configure Kernel Routing & Systemd

Enable IPv4 packet forwarding in the kernel so LR01 can route traffic between subnets:

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.d/99-ipforward.conf
sudo sysctl -p /etc/sysctl.d/99-ipforward.conf
```

Enable and start both WireGuard tunnels:

Bash

```
sudo systemctl enable --now wg-quick@wg0
sudo systemctl enable --now wg-quick@wg1
```

Step 5: Update `nftables` Rules

Allow incoming UDP traffic on ports `51820` and `51821`, and allow forwarding between `wg0`, `wg1`, and local LAN `10.101.50.0/24`.

Ensure your `/etc/nftables.conf` includes the following rules in the filter chains:

Code snippet

```
table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;

        iif "lo" accept
        ct state established,related accept

        # Allow SSH and ICMP
        tcp dport 22 accept
        icmp type echo-request accept

        # Allow Inbound WireGuard Tunnels on WAN
        udp dport { 51820, 51821 } accept
    }

    chain forward {
        type filter hook forward priority filter; policy drop;

        ct state established,related accept

        # Transit routing between CHQ (wg0) and CP (wg1)
        iifname "wg0" oifname "wg1" accept
        iifname "wg1" oifname "wg0" accept

        # Forwarding between HQ LAN (10.101.50.0/24) and VPN Tunnels
        ip saddr 10.101.50.0/24 oifname { "wg0", "wg1" } accept
        iifname { "wg0", "wg1" } ip daddr 10.101.50.0/24 accept

        # Allow WAN repository access (203.0.113.20)
        ip daddr 203.0.113.20 accept
    }
}
```

Reload `nftables` to apply changes:

Bash

```
sudo systemctl reload nftables
```

Step 6: Verify Tunnels & Connectivity

Bash

```
# Check status of WireGuard interfaces and active peers
sudo wg show

# Check routing table
ip route show

# Test link to CHQ-LR endpoint IP
ping -c 2 10.250.101.21

# Test link to LR02 endpoint IP
ping -c 2 10.251.101.22
```

Why
AllowedIPs = 10.251.101.22/32
not
AllowedIPs = 10.251.101.22/30
???

Using `/32` instead of `/30` in the `AllowedIPs` peer section is a strict WireGuard design requirement for point-to-point links.

1. The Core Reason: WireGuard's Cryptokey Routing (CKR)

WireGuard uses **Cryptokey Routing**, which maps **one specific IP address directly to one specific public key**.

When a router sends a packet to a tunnel peer, WireGuard looks at `AllowedIPs` to decide **which public key to encrypt the packet for**.

- **If you use `10.251.101.22/32`:**
  WireGuard explicitly maps `10.251.101.22` to **LR02's Public Key**. Every packet sent to `.22` is encrypted with `.22`'s key. Clear and unambiguous.
- **If you use `10.251.101.22/30`:**
  A `/30` subnet covers four IP addresses (`10.251.101.20` to `10.251.101.23`). This includes **LR01's own local interface IP (`10.251.101.21`)**.

  If you tell LR01 that the peer owns the entire `/30` range, WireGuard gets confused because it thinks LR02 also owns LR01's local IP address (`.21`).

2. Breakdown of the `/30` Subnet

IP Address	Role in Tunnel	Belong to Peer (LR02)?
`10.251.101.20`	Network Identifier	No
`10.251.101.21`	**LR01 (Local Interface)**	**NO — This is LR01!**
`10.251.101.22`	**LR02 (Remote Peer)**	**YES**
`10.251.101.23`	Subnet Broadcast	No

3. Comparing the Configurations

❌ Incorrect Configuration (`/30`)

Ini, TOML

```ini
[Peer]
# Bad: Claims LR02 owns 10.251.101.20 through 10.251.101.23
AllowedIPs = 10.251.101.22/30, 10.101.51.0/24
```

Problem: Causes routing collisions and broken return paths because LR01's own IP (`.21`) falls inside the peer's `AllowedIPs` range.

✅ Correct Configuration (`/32` + Subnets)

Ini, TOML

```
[Peer]
# Good: Explicitly maps ONLY LR02's single host IP and its remote LAN
AllowedIPs = 10.251.101.22/32, 10.101.51.0/24
```

Result:

1. `10.251.101.22/32` → Routes traffic destined specifically for LR02's tunnel endpoint.
2. `10.101.51.0/24` → Routes traffic destined for the CommandPost (CP) LAN behind LR02.

Why different ports
ListenPort = 51821
vs
Endpoint = 203.0.113.166:51820
?

`ListenPort` is the port on **LR01 itself**, while `Endpoint` specifies the destination IP and port on **LR02**.

They do not need to match because they refer to two completely different sides of the network socket connection.

1. The Anatomy of the Two Settings

- `ListenPort = 51821` **(Local Setting on LR01):**
  Tells LR01's kernel which local UDP socket to open on LR01 to receive incoming packets for this specific interface (`wg1`).
- `Endpoint = 203.0.113.166:51820` **(Remote Setting pointing to LR02):**

Tells LR01 where to **send** outgoing encrypted UDP packets across the WAN to reach LR02.

2. Why LR01 Uses `51821` Locally for `wg1`

Because LR01 runs **two separate WireGuard tunnels simultaneously**:

1. `wg0` **(Tunnel to CHQ-LR):** Listens locally on port `51820`.
2. `wg1` **(Tunnel to LR02):** Listens locally on port `51821`.

Two different network interfaces on the same Linux host cannot bind to the same UDP port at the same time. To avoid a port collision on LR01, `wg0` uses `51820` and `wg1` uses `51821`.

3. Why `Endpoint` Uses `51820` on LR02

LR02 only runs a single WireGuard tunnel (`wg0` linking to LR01).

Since LR02 only has one tunnel interface, it binds to the default WireGuard UDP port `51820` on its own WAN interface (`203.0.113.166`).

Therefore, when LR01 sends packets to LR02, it must send them to destination port `51820`.

Summary Diagram

Plaintext

```
[  LR01  ]                                                    [  LR02  ]
203.0.113.162                                                203.0.113.166
+------------------+                                         +----------------
| interface wg0    |                                         | interface wg0
| ListenPort: 51820 | <--- Links to CHQ-LR                   | ListenPort: 5182
|                  |                                         |
| interface wg1    |                                         | Endpoint:
| ListenPort: 51821 | ------------ Outgoing UDP ------------> | 203.0.113.162:
|                  |   Destination: 203.0.113.166:51820      | 51821
+------------------+                                         +----------------
```

(Note: On LR02's `/etc/wireguard/wg0.conf`, its `Endpoint` back to LR01 will accordingly be set to 203.0.113.162:51821.)

accordingly be set to `203.0.113.162:51821`).

```
linux_admin@lr01:~$ sudo wg show
interface: wg0
  public key:
V3FKIJLUOyhyTL71gfG6vuU1apwHRKpzeE8NNuW(
```

Your WireGuard tunnel itself is healthy (`wg show` shows a successful handshake and `10.251.101.22` pings fine), but traffic into the **CommandPost (CP) LAN** (`10.101.51.0/24`) is being blocked or dropped.

The 4 most common reasons why `10.101.51.100` (**CP-APC**) fails to respond:

1. Windows Firewall on CP-APC (Most Common)

By default, Windows Defender Firewall on `10.101.51.100` blocks ICMP Echo Requests (pings) coming from external subnets (`10.101.50.0/24` or `10.251.101.0/30`).

Fix on CP-APC: Open PowerShell as Administrator and run:

PowerShell

```
New-NetFirewallRule -Name "Allow_ICMPv4_In" -DisplayName "Allow ICMPv4 Inbound"
```

2. Missing IP Forwarding on LR02

LR02 receives the ping packet on its `wg0` interface, but if IPv4 forwarding is disabled in the kernel, it will drop the packet instead of routing it to its local LAN (`ens3` / `eth0`).

Fix on LR02:

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.d/99-ipforward.conf
```

3. `nftables` Drop Rule on LR02

LR02's firewall might be dropping forwarded traffic originating from `wg0` and heading to the local LAN.

Fix on LR02: Check your `nftables` ruleset (`sudo nft list ruleset`). Ensure the `forward` chain accepts traffic from `wg0` to `10.101.51.0/24` :

Code snippet

```
# In /etc/nftables.conf under 'chain forward'
iifname "wg0" ip daddr 10.101.51.0/24 accept
ip saddr 10.101.51.0/24 oifname "wg0" accept
```

Reload rules:

Bash

```
sudo systemctl reload nftables
```

4. Missing Static Return Route on CP-APC

CP-APC has two NICs (LAN and extNet). Per lab rules, its Default Gateway points to **extNet**. If CP-APC does not have a static route for `10.101.50.0/24` pointing to `10.101.51.254` (LR02), reply traffic leaves through the wrong interface.

Fix on CP-APC: Open PowerShell as Administrator and run:

PowerShell

```
New-NetRoute -DestinationPrefix 10.101.50.0/24 -NextHop 10.101.51.254 -Interface
```

Quick Diagnostic Step

Run `tcpdump` on **LR02** while pinging `10.101.51.100` from **LR01**:

Bash

```
# On LR02:
sudo tcpdump -n -i any host 10.101.51.100 and icmp
```

- **If you see ICMP Request on `wg0` but NOT on LAN interface:** IP forwarding or `nftables` on **LR02** is blocking it.
- **If you see ICMP Request go out LAN interface, but NO ICMP Reply: CP-APC** Windows Firewall is dropping it.
- **If you see ICMP Reply sent toward extNet/wrong gateway: CP-APC** is missing its static return route.

i am on lr01

Since you are on **LR01**, you can use `tcpdump` directly to track down where the packet is getting lost or stuck.

Run a targeted packet capture on **LR01** in one terminal window, then trigger the ping in another:

Bash

```
sudo tcpdump -n -i any host 10.101.51.100 and icmp
```

How to Read the `tcpdump` Output

Scenario A: Packets leave `wg0`, but no replies return

Plaintext

```
IP 10.251.101.21 > 10.101.51.100: ICMP echo request
IP 10.251.101.21 > 10.101.51.100: ICMP echo request
```

- **Meaning:** LR01 is successfully encrypting and sending the ICMP Echo Request over `wg0` to LR02.
- **The issue is on LR02 or CP-APC:**
  1. **LR02 `nftables` or `ip_forward`:** LR02 receives the packet but drops it before forwarding to its local LAN.
  2. **CP-APC Windows Firewall:** `10.101.51.100` receives the ping but blocks ICMP inbound.
  3. **CP-APC Routing:** `10.101.51.100` sends the reply out its `extNet` interface instead of back to `10.101.51.254` (LR02).

Scenario B: Packets are dropped locally on LR01

Plaintext

```
(No output appears when you ping)
```

- **Meaning:** LR01's own `nftables forward` or `output` chain is blocking the packet before it enters `wg0`.
- **Fix on LR01:** Check `sudo nft list ruleset` and ensure `outbound` traffic to

`10.101.51.0/24` via interface `wg0` is permitted.

Next Action

Log into **LR02** (`10.101.51.254`) via SSH or Proxmox console and run:

Bash

```
# 1. Enable IP forwarding on LR02
sudo sysctl -w net.ipv4.ip_forward=1

# 2. Watch for pings arriving on LR02
sudo tcpdump -n -i any host 10.101.51.100 and icmp
```

проанализируй и объясни отличия

#!/usr/sbin/nft -f

Главное отличие приведенного конфига от предложенного ранее обобщенного варианта — **строгий минимализм (Zero Trust) и явное соответствие архитектуре лаборатории.**

Предоставленный конфигурационный файл написан с точной привязкой к конкретным интерфейсам (`ens3` — LAN, `ens4` — WAN) и IP-сетям **POD 05 (Purple).**

Сравнительный анализ ключевых отличий

Критерий	Ранее предложенный шаблон	Ваш предоставленный `nftables.conf`	Почему ваш в лучше/отлича

Имена интерфейсов	Использовались generic-имена (`<WAN_IF>` , `wg0` , `wg1`)	Явное указание `ens3` **(LAN)** и `ens4` **(WAN)**	Исключает ог автоопредел interfaces.
Порты WireGuard	Открывались два порта `udp dport { 51820, 51821 }`	`iifname "ens4" udp dport 51820 accept`	На WAN-инте `ens4` приним входящее подключение для одного ту (`51820`), так `wg0` идет на клиент.
Фильтрация Input	`iif "lo" accept` , общие правила для SSH	`iifname "ens3" ip saddr 10.101.50.0/24 tcp dport 22 accept`	**Безопасност** на роутер раз **только** из лок сети HQ (`10.101.50.0` через `ens3` . или VPN войт нельзя.
Фильтрация Forward	Обобщенные правила пропуска туннельного трафика	Строгие пары `iifname` **+** `oifname` **+** `ip saddr` **+** `ip daddr`	Разрешены то строго задан направления конкретными подсетями HQ CHQ и Reposi
Транзит CHQ ↔ CP	Были правила transit `wg0` ↔ `wg1`	**Отсутствует правило CP ↔ CHQ** *(критический момент)*	В данном фай трафик из CP (или наоборо **блокируется** так как нет пр `iifname "wg oifname "wg0`

Правила NAT	`ip daddr 203.0.113.20 oif "<WAN_IF>" masquerade`	`ip saddr 10.101.50.0/24 ip daddr 203.0.113.20 oifname "ens4" masquerade`	Маскарадинг ограничен ст источником `10.101.50.0` гарантируя отсутствие N туннельного WG.

Анализ специфических секций вашего конфига

1. Цепь `input` (Защита самого роутера LR01)

- `iifname "wg0" accept` и `iifname "wg1" accept` : Разрешает трафик, адресованный **непосредственно самому LR01** по его туннельным IP (например, для выполнения `ping 10.250.101.22` или `ping 10.251.101.21`).
- `ip protocol icmp accept` : Разрешает ICMP (ping) на сам роутер со всех интерфейсов.

2. Цепь `forward` (Транзитный трафик через LR01)

Каждое правило строго проверяет входной и выходной интерфейсы, а также source/destination IP:

- **HQ → CHQ:** Разрешает только из `10.101.50.0/24` (`ens3`) в `10.100.101.0/24` (`wg0`).
- **CHQ → HQ:** Разрешает обратный инициализированный трафик или новые запросы из `10.100.101.0/24` (`wg0`) в `10.101.50.0/24` (`ens3`).
- **HQ → CP:** Разрешает из `10.101.50.0/24` (`ens3`) в `10.101.51.0/24` (`wg1`).
- **CP → HQ:** Разрешает из `10.101.51.0/24` (`wg1`) в `10.101.50.0/24` (`ens3`).
- **HQ → Repository:** Разрешает пакеты из LAN HQ к Linux Repository (`203.0.113.20`) через WAN (`ens4`).

Возможная проблема в вашем конфиге (Если потребуется связь CP ↔ CHQ)

В таблице маршрутизации LR01 указан как транзитный узел между CP и CHQ

(`Router VPN destinations` в схеме задания: *CHQ-LR reaches 10.101.51.0/24 via LR01*).

Если узлам из **CP** (`10.101.51.0/24`) потребуется доступ к **CHQ** (`10.100.101.0/24`) (например, доступ с CP-APC к `www.chq.raccoon.lab / 10.100.101.20`), текущий ruleset их **заблокирует**, так как в цепи `forward` нет правил для связи `wg0` и `wg1` .

Если транзит CP ↔ CHQ необходим, добавьте в `chain forward` **:**

Code snippet

```
# CP -> CHQ via LR01
iifname "wg1" \
    oifname "wg0" \
    ip saddr 10.101.51.0/24 \
    ip daddr 10.100.101.0/24 \
    accept

# CHQ -> CP via LR01
iifname "wg0" \
    oifname "wg1" \
    ip saddr 10.100.101.0/24 \
    ip daddr 10.101.51.0/24 \
    accept
```

provide for lr02 wg0.conf and nftables.conf for double-check

Below are the complete, ready-to-use configuration files for **LR02** (POD 05 / CommandPost gateway).

1. `/etc/wireguard/wg0.conf`

Ini, TOML

```
[Interface]
# LR02 local point-to-point IP inside the tunnel subnet
Address = 10.251.101.22/30
# Local port listened to on LR02's WAN interface (ens4)
ListenPort = 51820
PrivateKey = <LR02_PRIVATE_KEY>

[Peer]
# Peer: LR01
PublicKey = <LR01_PUBLIC_KEY>
# Connect to LR01's WAN IP on the port specified for wg1 (51821)
Endpoint = 203.0.113.162:51821
# Routing allowed IPs: HQ LAN (10.101.50.0/24), CHQ LAN (10.100.101.0/24), and t
AllowedIPs = 10.101.50.0/24, 10.100.101.0/24, 10.251.101.20/30
PersistentKeepalive = 25
```

2. /etc/nftables.conf

Code snippet

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {

    chain input {
        type filter hook input priority filter;
        policy drop;

        # Loopback interface
        iifname "lo" accept

        # Connection tracking (Established / Related)
        ct state established,related accept

        # Allow ICMP (Ping)
        ip protocol icmp accept
```

```
        # SSH access strictly from CP LAN
        iifname "ens3" \
            ip saddr 10.101.51.0/24 \
            tcp dport 22 \
            accept

        # Incoming WireGuard connections from LR01 on WAN (ens4)
        iifname "ens4" udp dport 51820 accept

        # Accept traffic directed to LR02's tunnel interface endpoints
        iifname "wg0" accept
    }

    chain forward {
        type filter hook forward priority filter;
        policy drop;

        # Connection tracking for forwarded packets
        ct state established,related accept

        # CP LAN -> HQ LAN (via wg0)
        iifname "ens3" \
            oifname "wg0" \
            ip saddr 10.101.51.0/24 \
            ip daddr 10.101.50.0/24 \
            accept

        # HQ LAN -> CP LAN (via wg0)
        iifname "wg0" \
            oifname "ens3" \
            ip saddr 10.101.50.0/24 \
            ip daddr 10.101.51.0/24 \
            accept

        # CP LAN -> CHQ LAN (via LR01 transit)
        iifname "ens3" \
            oifname "wg0" \
            ip saddr 10.101.51.0/24 \
            ip daddr 10.100.101.0/24 \
            accept

        # CHQ LAN -> CP LAN (via LR01 transit)
        iifname "wg0" \
            oifname "ens3" \
            ip saddr 10.100.101.0/24 \
```

```
            ip daddr 10.101.51.0/24 \
            accept

        # CP LAN -> Repository (via WAN ens4)
        iifname "ens3" \
            oifname "ens4" \
            ip saddr 10.101.51.0/24 \
            ip daddr 203.0.113.20 \
            accept
    }

    chain output {
        type filter hook output priority filter;
        policy accept;
    }
}

table ip nat {

    chain postrouting {
        type nat hook postrouting priority srcnat;
        policy accept;

        # NAT only for traffic heading to Linux Repository via WAN
        ip saddr 10.101.51.0/24 \
            ip daddr 203.0.113.20 \
            oifname "ens4" \
            masquerade
    }
}
```

Essential Pre-Flight Checks on LR02

Before applying the configuration, ensure system-level settings are active:

1. **Enable Kernel IPv4 Forwarding:**

Bash

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.d/99-ipforward.conf
```

. **Reload Firewalls & Interfaces:**

Bash

```
sudo nft -f /etc/nftables.conf
sudo systemctl restart wg-quick@wg0
```

how to add persistent route on HQ-APC windows server
CP-APC ⟶ HQ-APC
I should have a persistent route for:
10.101.51.0/24

To add a persistent static route on **HQ-APC** (Windows Server) so it knows how to route return traffic back to the CommandPost LAN (`10.101.51.0/24`) via **LR01** (`10.101.50.254`), run PowerShell or Command Prompt with administrative privileges.

Option 1: PowerShell (Recommended)

Open **PowerShell as Administrator** and execute:

PowerShell

```
New-NetRoute -DestinationPrefix "10.101.51.0/24" -NextHop "10.101.50.254" -Inter
```

Note: Replace `"LAN"` *with the exact name of your LAN network interface on HQ-APC if it uses a different name (e.g.,* `"Ethernet 0"` *).*

Option 2: Command Prompt (`route.exe`)

Open **cmd.exe as Administrator** and execute:

DOS

```
route -p ADD 10.101.51.0 MASK 255.255.255.0 10.101.50.254
```

- The `-p` flag ensures the route remains persistent across system reboots.

Verification & Testing

1. **Verify the routing table:**

PowerShell

```
Get-NetRoute -DestinationPrefix "10.101.51.0/24"
```

or via CMD:

DOS

```
route print 10.101.51.*
```

2. **Test connectivity from HQ-APC:**

PowerShell

```
Test-NetConnection -ComputerName 10.101.51.100 -TraceRoute
```

Additional Windows Firewall Check

If HQ-APC still fails to respond to pings from CP-APC (`10.101.51.100`), ensure Windows Defender Firewall allows inbound ICMP from remote subnets:

PowerShell

```
Set-NetFirewallRule -Name "FPS-ICMP4-ERQ-In" -Enabled True -Direction Inbound
```

route -p ADD 10.101.51.0 MASK 255.255.255.0 10.101.50.254

how to delete this route?

To delete this route on **HQ-APC**, open **Command Prompt** or **PowerShell** as **Administrator** and run:

DOS

```
route DELETE 10.101.51.0
```

Alternative: PowerShell

If you prefer using native PowerShell cmdlets:

PowerShell

```
Remove-NetRoute -DestinationPrefix "10.101.51.0/24" -NextHop "10.101.50.254" -Co
```

Verification

To confirm the route has been removed, check the routing table:

DOS

```
route print 10.101.51.*
```

If deleted successfully, it should return `No routes found.`

4. Setting up time synchronization
Exercise 3 · Configure NTP on both routers
Configure
LR01 in HQ and LR02 in CP as NTP clients and serv

Here are the concrete `/etc/chrony/chrony.conf` configurations tailored specifically to **POD 05 (Purple)** based on your network setup.

1. LR01 Configuration (`/etc/chrony/chrony.conf`)

On **LR01**, replace or comment out existing `server` / `pool` entries with the following:

Ini, TOML

```
# Welcome to chrony configuration file for LR01 (HQ)

# Distribution defaults for drift and logging
driftfile /var/lib/chrony/chrony.drift
```

```
logdir /var/log/chrony

# Upstream NTP Server: CHQ-LR WAN IP (Classroom 1 · Raccoon)
server 203.0.113.10 iburst

# Bind NTP server service exclusively to HQ LAN IP
bindaddress 10.101.50.254

# Allow NTP requests from HQ LAN subnet
allow 10.101.50.0/24
```

2. LR02 Configuration (`/etc/chrony/chrony.conf`)

On **LR02**, replace or comment out existing `server` / `pool` entries with the following:

Ini, TOML

```
# Welcome to chrony configuration file for LR02 (CP)

# Distribution defaults for drift and logging
driftfile /var/lib/chrony/chrony.drift
logdir /var/log/chrony

# Upstream NTP Server: CHQ-LR WAN IP (Classroom 1 · Raccoon)
server 203.0.113.10 iburst

# Bind NTP server service exclusively to CP LAN IP
bindaddress 10.101.51.254

# Allow NTP requests from CP LAN subnet
allow 10.101.51.0/24
```

3. Critical Step: Disable Competing Services & Apply

Run these commands on **both** LR01 and LR02:

Bash

Bash

```
# Disable systemd-timesyncd if present
sudo systemctl stop systemd-timesyncd
sudo systemctl disable systemd-timesyncd

# Restart and enable chrony
sudo systemctl enable chrony
sudo systemctl restart chrony
```

4. Firewall Requirement (`nftables.conf`)

Ensure `nftables` on both routers allows **UDP 123** for local LAN clients in `chain input` :

On LR01:

Code snippet

```
# Allow NTP from HQ LAN
iifname "ens3" ip saddr 10.101.50.0/24 udp dport 123 accept
```

On LR02:

Code snippet

```
# Allow NTP from CP LAN
iifname "ens3" ip saddr 10.101.51.0/24 udp dport 123 accept
```

5. Verification Commands

- **Verify Chrony Status on Routers:**

Bash

```
chronyc sources -v
```

```
chronyc tracking
```

(Ensure `203.0.113.10` shows `^*` indicating active synchronization).

3. **Run Test from Windows Clients:**
   - **From HQ-APC:**

     DOS

     ```
     w32tm /stripchart /computer:10.101.50.254 /dataonly /samples:5
     ```

   - **From CP-APC:**

     DOS

     ```
     w32tm /stripchart /computer:10.101.51.254 /dataonly /samples:5
     ```

4. **Check Client Query Logs on Routers:**

   Bash

   ```
   chronyc clients
   ```

```
table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;
        iifname "lo" accept
```

To allow PING (ICMP Echo Request) originating from **LR02** (or passing through LR02's WireGuard interface `wg1`) directly into the local **HQ LAN** (`10.101.50.0/24`), add an explicit ICMP forward rule in the `forward` chain.

> ⚠ NTP Protocol Port Note
>
> Make sure you also allow **UDP port 123** on `ens3` in the `input` chain so local HQ hosts can reach Chrony on LR01 (`10.101.50.254`).

Add Rule via `nft` Command Line

Run this directly on **LR01**:

Bash

```
sudo nft add rule inet filter forward iifname "wg1" oifname "ens3" ip saddr 10.2
```

*(Note: If you want to allow PING from **any** host behind LR02's tunnel subnet rather than just LR02's point-to-point IP `10.251.101.22`, change `ip saddr 10.251.101.22` to `ip saddr 10.251.101.20/30` or `ip saddr 10.101.51.0/24`).*

Updated `/etc/nftables.conf`

Insert the new rule under the `# CP -> HQ` section of your `chain forward`:

Code snippet

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {
        chain input {
```

```
        type filter hook input priority filter; policy drop;

        iifname "lo" accept
        ct state established,related accept
        ip protocol icmp accept

        # SSH from HQ LAN
        iifname "ens3" ip saddr 10.101.50.0/24 tcp dport 22 accept

        # NTP from HQ LAN (Required for Chrony)
        iifname "ens3" ip saddr 10.101.50.0/24 udp dport 123 accept

        # WireGuard WAN ports
        iifname "ens4" udp dport 51820 accept
        iifname "ens4" udp dport 51821 accept

        # VPN tunnel endpoints
        iifname "wg0" accept
        iifname "wg1" accept
}

chain forward {
        type filter hook forward priority filter; policy drop;

        ct state established,related accept

        # HQ -> CHQ
        iifname "ens3" oifname "wg0" ip saddr 10.101.50.0/24 ip daddr 10

        # CHQ -> HQ
        iifname "wg0" oifname "ens3" ip saddr 10.100.101.0/24 ip daddr 1

        # HQ -> CP
        iifname "ens3" oifname "wg1" ip saddr 10.101.50.0/24 ip daddr 10

        # CP LAN -> HQ LAN
        iifname "wg1" oifname "ens3" ip saddr 10.101.51.0/24 ip daddr 10

        # LR02 (Tunnel IP) -> HQ LAN ICMP Ping
        iifname "wg1" oifname "ens3" ip saddr 10.251.101.20/30 ip daddr

        # HQ -> Repository
        iifname "ens3" oifname "ens4" ip saddr 10.101.50.0/24 ip daddr 2
}

chain output {
```

```
        chain output {
                type filter hook output priority filter; policy accept;
        }
}

table ip nat {
        chain postrouting {
                type nat hook postrouting priority srcnat; policy accept;

                # Only Repository access is NATed.
                ip saddr 10.101.50.0/24 ip daddr 203.0.113.20 oifname "ens4" mas
        }
}
```

Verification

1. Reload the ruleset:

Bash

```
sudo nft -f /etc/nftables.conf
```

2. Test ping from **LR02**:

Bash

```
ping 10.101.50.100
```

3. To verify rule counters on **LR01**:

Bash

```
sudo nft list chain inet filter forward -a
```

```
sudo nft list chain inet filter forward -a
Error: syntax error, options must be specified before
commands
nft list chain inet filter forward -a
  ^                 ~~
```

The `-a` flag (which shows handle numbers) is an option for the `nft` executable itself, so it must go **before** the command `list`.

Correct Syntax

Run the command with `-a` right after `nft`:

Bash

```
sudo nft -a list chain inet filter forward
```

Useful Related Commands

List the chain with line numbers/handles:

Bash

```
sudo nft -a list chain inet filter forward
```

List the entire table with handles:

Bash

```
sudo nft -a list table inet filter
```

Delete a specific rule using its handle (e.g., handle 12):

Bash

```
sudo nft delete rule inet filter forward handle 12
```

graylog search syntax
how to search
logger -p user.notice -t DS_ComplexLab "Graylog test from $(hostname) at $(date -Is)"

To find messages created by that exact `logger` command in Graylog, search using Graylog's standard search bar syntax (Lucene/Elasticsearch-based).

1. Most Direct & Precise Searches

Since `logger` sets the syslog `syslog_facility` and `syslog_program` fields automatically, search using those specific fields:

Search by Application Tag (`-t DS_ComplexLab`):

Plaintext

```
application_name:"DS_ComplexLab"
```

(Depending on your Graylog syslog input extractor, this field may also be named

`syslog_program:"DS_ComplexLab"` *or* `source_program:"DS_ComplexLab"` *).*

Search by Facility/Level (`-p user.notice` **):**

Plaintext

```
syslog_facility:"user" AND level:5
```

(In Graylog/syslog, `notice` *maps to Syslog Level* `5` *).*

2. Searching by Message Content or Host

Search by Exact String Fragment:

Plaintext

```
"Graylog test from"
```

Search by Hostname & Program:

Plaintext

```
source:"lr01" AND application_name:"DS_ComplexLab"
```

(Replace `lr01` *with the result of your* `$(hostname)` *).*

3. Combining Fields for Exact Match

To filter out any ambiguity:

Plaintext

```
application_name:"DS_ComplexLab" AND message:"Graylog test from"
```

Quick Syntax Rules Reference

Goal	Syntax Example
Exact phrase	`message:"Graylog test"`
Field wildcard	`application_name:DS_*`
Multiple conditions	`source:"lr01" AND level:5`
Exclude results	`application_name:"DS_ComplexLab" AND NOT level:3`

is there analog rsyslog fow windows server? how to send messages to graylog from windows server?

Yes, there are several reliable options for sending Windows event logs to Graylog.

1. Equivalent Tools for Windows

While `rsyslog` is Linux-native, Windows uses the **Windows Event Log** system instead of plain text `/var/log` files. To forward these logs to Graylog, you use a dedicated log shipper service:

- **NXLog Community Edition (Most Popular `rsyslog` Direct Equivalent):** A lightweight, extremely flexible log forwarder that runs as a Windows service. It converts Windows Event Logs into Syslog or GELF (Graylog Extended Log Format) and sends them over UDP/TCP/TLS.
- **Graylog Sidecar + Filebeat/Winlogbeat (Official Graylog Approach):** Graylog's recommended solution. You install the lightweight **Winlogbeat** agent alongside **Graylog**

Sidecar, allowing you to manage Windows log collection configurations centrally from the Graylog web UI.

- **Fluentbit / Logstash:** Heavier alternatives capable of forwarding Windows Event Logs.

2. Method A: Using NXLog (Quickest Setup)

Step 1: Install NXLog on Windows Server

Download and install **NXLog Community Edition** on the Windows Server.

Step 2: Configure `C:\Program Files\nxlog\conf\nxlog.conf`

Replace or update the configuration file with the following setup to send logs in **GELF** (recommended) or **Syslog** format:

Ini, TOML

```
PanicSoft /var/log/nxlog/nxlog.log
# Set installation path
define ROOT C:\Program Files\nxlog

Moduledir %ROOT%\modules
CacheDir  %ROOT%\data
Pidfile   %ROOT%\data\nxlog.pid
SpoolDir  %ROOT%\data
LogFile   %ROOT%\data\nxlog.log

<Extension gelf>
    Module      xm_gelf
</Extension>

# Input: Collect Windows Event Logs (Application, System, Security)
<Input in_eventlog>
    Module      im_msvistalog
</Input>

# Output: Send to Graylog Server via UDP GELF
<Output out_graylog>
    Module      om_udp
    Host        <GRAYLOG_SERVER_IP>
    Port        12201
    OutputType  GELF
```

```
</Output>

<Route 1>
    Path        in_eventlog => out_graylog
</Route>
```

Step 3: Start the Service

Open PowerShell as Administrator:

PowerShell

```
Start-Service nxlog
Set-Service nxlog -StartupType Automatic
```

3. Method B: Command-Line Log Generation (Like `logger`)

If you want a direct PowerShell equivalent to the Linux `logger` command to send custom one-off messages to Graylog without installing a full service:

Send UDP Syslog Directly from PowerShell

PowerShell

```
$GraylogIP = "<GRAYLOG_SERVER_IP>"
$Port = 514
$Message = "<13>1 $(Get-Date -Format o) $env:COMPUTERNAME DS_ComplexLab - - - Gr

$UdpClient = New-Object System.Net.Sockets.UdpClient
$EncodedMsg = [System.Text.Encoding]::UTF8.GetBytes($Message)
$UdpClient.Send($EncodedMsg, $EncodedMsg.Length, $GraylogIP, $Port)
$UdpClient.Close()
```

Write to Windows Event Log (Picked up by NXLog/Winlogbeat)

PowerShell

```powershell
# Create source if it doesn't exist
if (![System.Diagnostics.EventLog]::SourceExists("DS_ComplexLab")) {
    New-EventLog -LogName Application -Source "DS_ComplexLab"
}

# Write test message
Write-EventLog -LogName Application -Source "DS_ComplexLab" -EntryType Informati
```

4. Required Input in Graylog

On your Graylog Server, navigate to **System** → **Inputs** and launch an input matching your shipper:

- For **NXLog GELF Output**: Launch a **GELF UDP** input on port `12201`.
- For **Syslog**: Launch a **Syslog UDP** input on port `514` or `1514`.

PS C:\WINDOWS\system32> powershell -ExecutionPolicy Bypass -File "c:\temp\test_graylog.ps1"
97

how to find in graylog by "Graylog test from PowerShell"?

To find log messages sent by your PowerShell script in Graylog, use Graylog's search bar with the following query syntax:

1. Direct Search Options

Search by Exact Phrase:

Search by Exact Phrase:

Plaintext

```
"Graylog test from PowerShell"
```

Search by Message Keyword:

Plaintext

```
message:"Graylog test"
```

Search by Hostname & Content:

Plaintext

```
source:"HQ-APC" AND message:"PowerShell"
```

(Replace `HQ-APC` with your actual Windows computer name).

2. Time Range & Field Filtering

Because your script returned `97` (indicating 97 bytes were successfully transmitted over UDP), make sure to adjust the **Time Range** picker in the top-left corner of Graylog:

1. Set the search filter to **Search in the last 15 minutes** (or **Relative → 1 Hour**).
2. If you are logging via a standard Syslog UDP input, filter by the application tag:

Plaintext

```
application_name:"DS_ComplexLab"
```

Troubleshooting If No Results Appear

Troubleshooting: If No Results Appear

If searching `"Graylog test"` returns no messages:

1. **Verify Graylog Input:** Go to **System** → **Inputs** in the Graylog web interface and confirm that a **Syslog UDP** input is running on port `514` (or whichever port `test_graylog.ps1` used). Check that the **Network IO** counter for that input increases when you execute the script.
2. **Check Router / Firewall Forwarding:** Ensure `nftables` on LR01 or LR02 permits UDP traffic originating from your Windows host (`10.101.50.100 / 10.101.51.100`) heading toward the Graylog server's IP and port.

```
$GraylogIP = "<GRAYLOG_SERVER_IP>"
$Port = 514
$Message = "<13>1 $(Get-Date -Format o)
$env:COMPUTERNAME DS_ComplexLab - - - Grayl
```

Here is the updated PowerShell script wrapped in a `try...catch` block. It explicitly catches socket and general exceptions, outputs useful diagnostic info to the console, and properly disposes of the UDP client object even if an error occurs.

PowerShell

```powershell
$GraylogIP = "203.0.113.20" # Replace with your actual Graylog server IP
$Port = 514
$Message = "<13>1 $(Get-Date -Format o) $env:COMPUTERNAME DS_ComplexLab - - - Gr

$UdpClient = $null

try {
    Write-Host "Attempting to send UDP message to ${GraylogIP}:${Port}..." -Fore

    $UdpClient = New-Object System.Net.Sockets.UdpClient
    $EncodedMsg = [System.Text.Encoding]::UTF8.GetBytes($Message)
```

```
    $EncodedMsg = [System.Text.Encoding]::UTF8.GetBytes($Message)

    # Send returns the number of bytes sent
    $BytesSent = $UdpClient.Send($EncodedMsg, $EncodedMsg.Length, $GraylogIP, $P

    Write-Host "SUCCESS: Successfully sent $BytesSent bytes to Graylog." -Foregr
}
catch [System.Net.Sockets.SocketException] {
    Write-Error "SOCKET ERROR: Failed to communicate with ${GraylogIP}:${Port}.
}
catch {
    Write-Error "GENERAL ERROR: An unexpected error occurred. Details: $($_.Exce
}
finally {
    if ($null -ne $UdpClient) {
        $UdpClient.Close()
        $UdpClient.Dispose()
    }
}
```

Key Improvements

- `finally` **Block:** Ensures `$UdpClient.Close()` and `$UdpClient.Dispose()` execute regardless of whether the script succeeds or throws an exception, preventing port/socket leaks.
- **Specific Exception Handling:** Separate `catch` block for `SocketException` to isolate network/IP resolution issues.
- **Informative Output:** Uses `Write-Host` and `Write-Error` with color coding so you immediately know whether execution succeeded or failed when running via PowerShell.

PS C:\WINDOWS\system32> Test-NetConnection 10.101.50.40 -Port 9000

The message is not appearing in Graylog because the **UDP protocol is fire-and-forget**.

`SUCCESS: Successfully sent 97 bytes` only means the local Windows socket successfully pushed the network packet out of your network interface (`10.101.50.100`). It **does not confirm** that Graylog received, processed, or indexed it.

Here are the most common causes and how to resolve them:

1. Graylog Input Configuration Mismatch (Most Common)

Your test script is sending to port `1514` , but your `Test-NetConnection` test was on port `9000` (which is Graylog's web interface/REST API, not a syslog input).

- **Input Port & Type:** Verify in Graylog (**System → Inputs**) that you have an active Input listening specifically on **UDP port 1514**.
- **Input Format:** Ensure the input type matches what your script sent:
  - If sending raw text/Syslog, your input must be **Syslog UDP** or **Raw/Plaintext UDP**.
  - If sending GELF formatted JSON, your input must be **GELF UDP**. If you send raw text to a GELF input, Graylog drops the packet silently.

2. Firewall / Router Dropping UDP 1514

While TCP port 9000 reached the host, UDP port 1514 might be blocked:

- **Host Firewall (Graylog Server):** Ensure `ufw` or `firewalld` on the Linux server running Graylog allows incoming traffic on `1514/udp` .
- **Privileged Ports:** If the input is set to port 514 instead of 1514, Linux prevents non-root processes (like the `graylog` user) from binding to ports below 1024 without `setcap` or iptables port redirection.

3. Time Sync / Timestamp Misalignment

Graylog sets the search time window to **"Search in the last 5 minutes"** by default.

- If your script sends a message with an explicit timestamp that is slightly off (or in UTC vs. local time), or if the Graylog server's system clock isn't synchronized via NTP, the log might be indexed in the **past or future**.
- **Fix:** Change the search time picker in Graylog from relative ("Search in the last 5 minutes") to **Absolute Range** or **"Search in all messages"**, then search again for `PowerShell` .

4. Verification & Troubleshooting Steps

① Check Active Graylog Inputs

Graylog Web UI

Go to **System / Inputs**. Confirm an input is running on **UDP 1514** and check the **Network IO** graph for that input to see if any bytes arrived when you ran the script.

Verification: If byte counters stay at zero during script execution, the packets are being dropped by a network/host firewall before reaching Graylog.

② Verify Incoming Packets on Server

Linux Shell on 10.101.50.40

Run `tcpdump` on the Graylog server while running your PowerShell script to see if UDP packets physically reach the network interface:

Bash

```
sudo tcpdump -i any udp port 1514 -vv -X
```

Verification: If `tcpdump` shows incoming UDP packets from `10.101.50.100`, the network delivery is successful and the issue lies inside Graylog (parsing errors or index delays).

③ Inspect Graylog Server Logs

Graylog Logs

If packets arrive but Graylog drops them (e.g., due to invalid GELF formatting), check Graylog's internal log file:

Bash

```
sudo tail -f /var/log/graylog-server/server.log
```

Verification: Look for `UncompressedGelfInput` or parsing exceptions occurring at the exact time you execute the script.

Input Type Syslog TCP

Switching from **Syslog UDP** to **Syslog TCP** changes the delivery mechanics fundamentally: TCP establishes a reliable connection and acknowledges received data, so any connection failures or port blocks will raise an immediate error in your PowerShell script rather than failing silently.

Step 1: PowerShell Script for Syslog TCP (Port 1514)

Run this updated PowerShell script on **HQ-APC** (`10.101.50.100`) to send your syslog message over a TCP stream:

PowerShell

```
$GraylogIP = "10.101.50.40"
$Port = 1514$Message = "<13>1 $(Get-Date -Format o)$env:COMPUTERNAME DS_ComplexL

$TcpClient = $null
$Stream = $null

try {
    Write-Host "Connecting to Graylog Syslog TCP endpoint at ${GraylogIP}:${Port

    # Establish TCP Connection
    $TcpClient = New-Object System.Net.Sockets.TcpClient
    $TcpClient.Connect($GraylogIP, $Port)

    if ($TcpClient.Connected) {
        Write-Host "SUCCESS: Connected to Graylog over TCP." -ForegroundColor Gr
```

```
        $Stream = $TcpClient.GetStream()
        $EncodedMsg = [System.Text.Encoding]::UTF8.GetBytes($Message)

        # Write message bytes to the TCP socket stream
        $Stream.Write($EncodedMsg, 0, $EncodedMsg.Length)
        $Stream.Flush()

        Write-Host "SUCCESS: Sent $($EncodedMsg.Length) bytes to Graylog via TCP
    }
}
catch [System.Net.Sockets.SocketException] {
    Write-Error "CONNECTION REFUSED / TIMEOUT: Could not establish TCP connectio
}
catch {
    Write-Error "ERROR: An unexpected error occurred. Details: $($_.Exception.Me
}
finally {
    if ($null -ne $Stream) { $Stream.Close(); $Stream.Dispose() }
    if ($null -ne $TcpClient) { $TcpClient.Close(); $TcpClient.Dispose() }
}
```

Important Note on Delimiters: Syslog TCP inputs expect a framing character—usually a newline (`\n` or `` `n `` in PowerShell)—to mark the end of a frame. Adding `` `n `` ensures Graylog immediately parses and indexes the log line.

Step 2: Ensure Graylog Input Configuration Matches

In the Graylog Web Interface (**System** → **Inputs**):

1. Verify that your input type is configured as **Syslog TCP** (not Syslog UDP or GELF TCP).
2. Confirm the listening port matches (e.g., `1514`).
3. Ensure **Bind address** is set to `0.0.0.0` (or `10.101.50.40`), not `127.0.0.1` .

Step 3: Verification Commands

1. Test TCP Port Open Status from PowerShell

PowerShell

```
Test-NetConnection 10.101.50.40 -Port 1514
```

Expected Result: `TcpTestSucceeded : True`

2. Monitor Incoming TCP Traffic on Graylog Host

Run `tcpdump` on the Graylog server (`10.101.50.40`) to watch the TCP handshake (`SYN` , `ACK`) and message transfer:

Bash

```
sudo tcpdump -i any tcp port 1514 -vv -A
```

3. Search in Graylog

Once the script returns `SUCCESS` , query Graylog's search bar:

Plaintext

```
"Graylog test from PowerShell over TCP"
```

It works!!!

Awesome! Glad to hear it's working! 🎉

Let me know if you run into anything else or need to tweak the setup down the road.

```
linux_admin@hq-srv:~/graylog$ ip route get 10.101.50.40
local 10.101.50.40 dev lo src 10.101.50.40 uid 1000
    cache <local>
linux_admin@hq-srv:~/graylog$ chronyc sources -
```

The issue is that `hq-srv` (`10.101.50.40`) **is trying to query** `203.0.113.10` **directly**, but according to your lab task, **only LR01 and LR02 should query** `203.0.113.10` **directly**. Internal hosts like `hq-srv` must query their local router (**LR01 at** `10.101.50.254`).

Additionally, because `bindaddress 10.101.50.254` was left inside `/etc/chrony/chrony.conf` on `hq-srv`, Chrony failed to bind to its own loopback/local IP address (`10.101.50.40`).

Root Cause Analysis

1. `^? 203.0.113.10` **with** `Reach = 0`: `hq-srv` is sending UDP 123 packets directly to WAN IP `203.0.113.10`. LR01's firewall or NAT rules block outgoing client NTP to WAN.
2. `bindaddress 10.101.50.254` **on** `hq-srv`: This directive tells Chrony to listen *only* on `10.101.50.254`. Since `10.101.50.254` belongs to LR01 (not `hq-srv`), Chrony on `hq-srv` cannot bind properly.
3. **Leftover Default Sources**: Lines like `sourcedir /run/chrony-dhcp` and `sourcedir /etc/chrony/sources.d` may load unwanted distribution pool sources.

Fix: Reconfigure Chrony on `hq-srv`

1. Edit `/etc/chrony/chrony.conf` on `hq-srv`

Open the configuration file:

Bash

```
sudo nano /etc/chrony/chrony.conf
```

Replace or update the directives so `hq-srv` points to **LR01 (** `10.101.50.254` **)**:

Ini, TOML

```
# Comment out DHCP and default sources directories to avoid extra pools
# sourcedir /run/chrony-dhcp
# sourcedir /etc/chrony/sources.d

# System defaults
driftfile /var/lib/chrony/chrony.drift
logdir /var/log/chrony
maxupdateskew 100.0
rtcsync
makestep 1 3

# Upstream NTP Server: Local Gateway LR01
server 10.101.50.254 iburst

# Remove 'bindaddress' and 'allow' lines on hq-srv unless hq-srv itself is actin
```

(Note: Remove `bindaddress 10.101.50.254` *and* `allow 10.101.50.0/24` *from* `hq-srv` *. Those lines belong only on LR01).*

2. Restart Chrony & Force Step Update

Run the following commands on `hq-srv` :

Bash

```
# Restart Chrony service
sudo systemctl restart chrony

# Force immediate time synchronization step
sudo chronyc makestep
```

Verification

1. Check Sources Status

Bash

```
chronyc sources -v
```

Expected Output: The source IP should now be `10.101.50.254`, with `MS` showing `^*`:

Plaintext

```
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^* 10.101.50.254                 2   6   377    12   +120us[ +150us] +/- 12ms
```

2. Check Tracking Info

Bash

```
chronyc tracking
```

Expected Output:

- `Reference ID : 0A6532FE` (or `10.101.50.254`)
- `Stratum : 2` or `3`
- `Leap status : Normal`

Summary Checklist for the Lab Topology

Host	Upstream Server	bindaddress	allow

	(server)		
LR01	203.0.113.10	10.101.50.254	10.101.50.0
LR02	203.0.113.10	10.101.51.254	10.101.51.0
hq-srv / HQ-APC	10.101.50.254	*(none / default)*	*(none)*
CP-APC	10.101.51.254	*(none / default)*	*(none)*

```
linux_admin@hq-srv:~/graylog$ sudo chronyc makestep
200 OK
linux_admin@hq-srv:~/graylog$ chronyc sources -v
```

The output shows that `hq-srv` is now contacting LR01 (`_gateway = 10.101.50.254`), but two indicators explain why it hasn't fully synchronized yet:

1. `Reach = 3` : `hq-srv` has only received its first 2 valid NTP responses (octal `3` = binary `00000011`). Chrony requires 3 to 4 consecutive successful polls (`Reach = 17` or `377`) to trust the clock and calculate offset drift.
2. `Stratum 3` **on LR01:** LR01 itself is reporting Stratum 3, meaning LR01 is active and properly serving time.

Step 1: Force Immediate Step & Reload

Since Chrony only polled twice, trigger a full sequence of 4 rapid measurement bursts:

Bash

```
sudo chronyc burst 4/4
```

Wait 5 seconds, then force the clock step again:

Bash

```
sudo chronyc makestep
```

Step 2: Verification

Check the sources register again:

Bash

```
chronyc sources -v
```

Expected Result:

- Mode/State should change from `^?` to `^*` (indicating current best active source).
- `Reach` will increase to `17` or higher.

Plaintext

```
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^* _gateway                      3   6    17     2   -120us[ -120us] +/- 2199us
```

Check tracking status:

Bash

```
chronyc tracking
```

Expected Result:

- `Reference ID : 0A6532FE` (or `10.101.50.254`)
- `Stratum : 4`
- `Leap status : Normal`

Verification Command

Run this command on `hq-srv` to verify that LR01 responds to 5 consecutive queries:

Bash

```
chronyc clients
```

*(Run on **LR01** to confirm `10.101.50.40` is registered as a querying client).*

```
linux_admin@hq-srv:~/graylog$ sudo chronyc burst 4/4
200 OK
linux_admin@hq-srv:~/graylog$ sudo chronyc makestep
200 OK
```

Look closely at the source state marker in `chronyc sources`:

Plaintext

```
^- _gateway                     3   6   377    22   -506ms[ -506ms] +/- 1682us
```

The symbol `^-` means `_gateway` (`10.101.50.254`) is valid and reachable (`Reach = 377`), but chrony considers it **"not combined" / rejected** because the offset between your server's current clock and LR01 is **-506ms**, exceeding the maximum allowed adjustment or skew threshold.

Step 1: Force Synchronization via `chronyc selectdata / makestep`

Run the following commands on `hq-srv` to force chrony to re-evaluate and step the clock immediately:

Bash

```
# Force chrony to ignore max-offset restrictions and step the system clock
sudo chronyc -a makestep
```

If `chronyc -a makestep` still leaves `Leap status: Not synchronised` , check if `maxupdateskew` or `maxdistance` is blocking selection in `/etc/chrony/chrony.conf` :

Bash

```
# Temporarily step the clock manually using date or ntpdate equivalent if skew i
sudo systemctl stop chrony
sudo chronyd -q 'server 10.101.50.254 iburst'
sudo systemctl start chrony
```

Step 2: Verification

1. **Check sources state:**

Bash

```
chronyc sources -v
```

Verify that `^-` *has changed to* `^*` *(active best source).*

. **Check tracking status:**

Bash

```
chronyc tracking
```

Verify that `Reference ID` *shows* `10.101.50.254` *(or hex equivalent),* `Stratum` *is* `4` *, and* `Leap status` *is* `Normal` *.*

```
linux_admin@hq-srv:~/graylog$ sudo systemctl stop chrony
linux_admin@hq-srv:~/graylog$ sudo chronyd -q 'server
10.101.50.254 iburst'
2026-09-15T14:22:26Z chronyd version 4.8 starting
```

The 0.507s clock step fixed the offset issue (`+217us` vs the previous `-506ms`).

In your `chronyc sources` output:

- `Reach = 1` : `chronyd` has only received **1 single response** since restarting (`systemctl start chrony`).
- `^?` : Chrony requires at least 3 valid samples to compute jitter, select the source, and transition from `^?` to `^*` .
- `Leap status: Not synchronised` : Remains unsynchronized until source selection completes.

Step 1: Force a Burst to Complete Selection Immediately

Run this burst command to send 4 fast NTP queries right now instead of waiting minutes for normal polling intervals:

Bash

```
sudo chronyc burst 4/4
```

Wait 3–5 seconds for the responses, then check sources again:

Bash

```
chronyc sources -v
```

Step 2: Final Verification

Check tracking status:

Bash

```
chronyc tracking
```

Expected Result:

- `MS Name/IP address` : Should show `^* _gateway` (`^* 10.101.50.254`).
- `Stratum : 4` (one level below LR01's Stratum 3).
- `Leap status : Normal` .
- `Reference ID : 0A6532FE` (hex for `10.101.50.254`).

```
linux_admin@hq-srv:~/graylog$ sudo chronyc burst 4/4
200 OK
linux_admin@hq-srv:~/graylog$ chronyc sources -v
```

`_gateway` is now fully reachable (`Reach = 77`), and the clock offset is tiny (`-483us`), but Chrony is still marking it as `^-` (valid candidate, not selected) and leaving tracking at `Stratum 0` .

This happens when the upstream server (LR01) has a high **root distance** (root delay + root dispersion > 1.5s), or Chrony's selection filters require explicit clock steps/re-evaluation.

Step 1: Force Step & Source Selection

Force Chrony to accept the source and step immediately:

Bash

```
sudo chronyc makestep
```

If it still stays on `^-` , increase `maxdistance` temporarily in `/etc/chrony/chrony.conf` on `hq-srv` or pass a force command:

Bash

```
# Append maxdistance directive if LR01's root dispersion is high
echo "maxdistance 16.0" | sudo tee -a /etc/chrony/chrony.conf
sudo systemctl restart chrony
sudo chronyc burst 4/4
```

Step 2: Verification

Check sources and tracking again:

Bash

```
chronyc sources -v
chronyc tracking
```

Expected Result:

- `MS` status changes to `^* _gateway .`
- `Stratum : 4 .`
- `Leap status : Normal .`

linux_admin@hq-srv:~/graylog$ sudo chronyc makestep
200 OK
linux_admin@hq-srv:~/graylog$ sudo nano /etc/chrony/chrony.conf

The configuration issue on **hq-srv** is clear: the server line in `/etc/chrony/chrony.conf` is missing the `iburst` option, and the upstream server on `10.101.50.254` (LR01) likely has its own synchronization/dispersion delay preventing auto-selection.

Additionally, chrony is pulling sources dynamically from `/run/chrony-dhcp` or `/etc/chrony/sources.d`, which can cause source evaluation conflicts.

Step 1: Explicitly Add `iburst` and `prefer`

Edit `/etc/chrony/chrony.conf` on `hq-srv` to force immediate synchronization and prioritize the gateway:

Bash

```
sudo nano /etc/chrony/chrony.conf
```

Find the `server 10.101.50.254` line near the bottom and change it to:

Plaintext

```
server 10.101.50.254 iburst prefer
```

Save and exit (`Ctrl+O` , `Enter` , `Ctrl+X` *).*

Step 2: Restart and Verify Selection

Restart `chronyd` and force a burst request:

Bash

```
sudo systemctl restart chrony
sudo chronyc burst 4/4
```

Check source status to confirm it transitions from `^-` to `^*` :

Bash

```
chronyc sources -v
```

Verification: The line should now read `^* _gateway` (or `^* 10.101.50.254`).

Step 3: Check Tracking Status

Run tracking to confirm full synchronization:

Bash

```
chronyc tracking
```

Verification:

- `Stratum : 4`
- `Leap status : Normal`
- `Reference ID : 0A6532FE ( 10.101.50.254 )`

If `_gateway` remains stuck on `^-` after adding `iburst prefer`, the root delay/dispersion on LR01 (`10.101.50.254`) itself is out of spec.

linux_admin@hq-srv:~/graylog$ sudo nano /etc/chrony/chrony.conf
linux_admin@hq-srv:~/graylog$ sudo systemctl restart chronyhrony/chrony.conf

The root cause of this state is that `_gateway` **(LR01) itself is unsynchronized** or reporting a **high root distance / leap status error**.

Because `chronyd` on `hq-srv` sees `_gateway` as valid (`Reach = 377`, `Offset = -2.4ms`), it marks it as candidate state `^-`. However, `chronyd` will refuse to select

`_gateway` with `^*` (which leaves `hq-srv` at `Stratum 0` and `Not synchronised`) if `_gateway` advertises that its own root clock is untrusted.

Step 1: Diagnose the Upstream Gateway (`_gateway`)

Run this command on `hq-srv` to inspect the exact NTP status flags returned by `_gateway` :

Bash

```
chronyc ntpdata
```

Look at two specific lines in the output:

- `Leap status` : If it says `Not synchronised` or `3 (unsynchronised)` , `_gateway` is refusing to serve authoritative time. Chrony
- `Root dispersion` / `Root delay` : If these values are excessively large (e.g., > 1.0 second), `hq-srv` will filter it out. Chrony

Step 2: Override Upstream Validation (If LR01 is an Isolated/Lab Clock)

If `10.101.50.254` is an isolated router/gateway in your environment that serves local system time without a higher upstream clock, force `hq-srv` to trust it regardless of its root distance or leap status:

1. Open `/etc/chrony/chrony.conf` : chrony – chrony.conf(5)

Bash

```
sudo nano /etc/chrony/chrony.conf
```

2. Modify the `server` line to include `trust` along with `prefer iburst` :

Plaintext

```
server 10.101.50.254 iburst prefer trust
```

. Restart and re-burst `chronyd` :

Bash

```
sudo systemctl restart chrony
sudo chronyc burst 4/4
```

Step 3: Verify Clock Synchronization

Check sources and tracking to confirm transition:

Bash

```
chronyc sources -v
chronyc tracking
```

Expected Result:

- `sources` : Shows `^* _gateway` (or `^* 10.101.50.254`). Chrony
- `tracking` : Chrony
  - `Stratum` : `4` (or `4` if LR01 is `Stratum 3`). Chrony
  - `Leap status` : `Normal` . Chrony + 1

linux_admin@hq-srv:~/graylog$ sudo chronyc ntpdata

Remote address : 10.101.50.254 (0A6532FE)
Remote port : 123

Remote port . 123

`10.101.50.254` is operating as a healthy **Stratum 3** NTP server (`Leap status: Normal` , passes all NTP tests `111 111 1111` , minimal root delay/dispersion).

`chronyd` on `hq-srv` is refusing to select `10.101.50.254` because it is processing duplicate or conflicting directives from the dynamic source files included at the top of `/etc/chrony/chrony.conf` (`sourcedir /run/chrony-dhcp` or `/etc/chrony/sources.d`), or because it lacks the `trust` option.

Step 1: Clean Up `/etc/chrony/chrony.conf`

Edit `/etc/chrony/chrony.conf` on `hq-srv` :

Bash

```
sudo nano /etc/chrony/chrony.conf
```

. **Comment out** or remove the dynamic source lines at the top:

Plaintext

```
# sourcedir /run/chrony-dhcp
# sourcedir /etc/chrony/sources.d
```

. Update the `server` line at the bottom to explicitly trust and prefer the gateway:

Plaintext

```
server 10.101.50.254 iburst prefer trust
```

(Save with `Ctrl+O` *, press* `Enter` *, exit with* `Ctrl+X` *.)*

Step 2: Restart and Force Clock Step

Restart `chronyd` and force an immediate step and burst:

Bash

```
sudo systemctl restart chrony
sudo chronyc makestep
sudo chronyc burst 4/4
```

Step 3: Verify Clock Synchronization

Check sources and tracking:

Bash

```
chronyc sources -v
chronyc tracking
```

How to verify success:

- In `chronyc sources`, the symbol before `_gateway` or `10.101.50.254` must be `^*` (not `^-`).
- In `chronyc tracking`, `Stratum` should be `4` and `Leap status` should report `Normal`.

how to get wireguard version?

To check the version of WireGuard installed on your system, use one of the following methods depending on your operating system and tools:

Linux

1. Check via the `wg` command line tool:

Bash

```
wg --version
```

(Shows the version of the `wireguard-tools` user-space utility).

2. Check kernel module version (if using kernel-space WireGuard):

Bash

```
modinfo wireguard | grep -i version
```

3. Check via package manager:

Debian / Ubuntu:

Bash

```
dpkg -s wireguard-tools
```

RHEL / CentOS / Fedora:

Bash

```
rpm -qa | grep wireguard
```

Windows

1. Command Prompt / PowerShell:
If the WireGuard binary folder is in your environment PATH, run:

PowerShell

```
wireguard --version
```

2. GUI:
Open the **WireGuard** desktop app and click the **About** button at the bottom right to view the client and driver versions.

macOS

1. Terminal (via Homebrew):

Bash

```
wg --version
```

2. macOS App Store Client:
Open the WireGuard app from the menu bar → **About WireGuard** to inspect the version release.

there is a warning in the checkmk
how to fix it?

how to fix it?

Site aliasLocal site cmkHost nameHQ-SRVService

The **WARNING** state is triggered by the **Check_MK Discovery** service because it detected a new network interface (`lnx_if: Interface 4`) on **HQ-SRV** that is not currently configured in Checkmk's monitored services list.

To resolve this warning, you need to update the service configuration for **HQ-SRV** in Checkmk so it either begins monitoring the new interface or ignores it.

Solution 1: Accept the New Interface into Monitoring (Web UI)

1. **Navigate to HQ-SRV Host Setup**

   Search for HQ-SRV in Checkmk

   Go to **Setup** → **Hosts** (or use the global search bar in Checkmk) and search for **HQ-SRV**.

2. **Open Service Configuration**

   View discovered services

   Click on **HQ-SRV** and select **Run service discovery** (or click the **Service discovery** icon next to the host name).

3. **Fix Missing Services**

   Add lnx_if Interface 4

   Under **Undecided services**, you will see `Interface 4` highlighted. Click **Fix All** or click **Accept all** at the top of the page to add the new interface to monitored services.

4. **Activate Changes**

   Commit changes in Checkmk

   Click the **Pending Changes** button (yellow marker at the top right of the screen) and click **Activate on selected sites**.

Verification: The **Check_MK Discovery** service on **HQ-SRV** will automatically clear and return to **OK** on its next scheduled run.

Solution 2: Fix via Command Line (Checkmk CLI)

If you have SSH access to your Checkmk server (running under the site user instance `cmk`):

1. Switch to the Checkmk site user:

Bash

```
sudo omd su cmk
```

2. Update service discovery for **HQ-SRV**:

Bash

```
cmk -I HQ-SRV
```

3. Reload/Activate the configuration:

Bash

```
cmk -R
```

Solution 3: Prevent Future Warnings (Automate Discovery)

If network interfaces frequently change in your lab environment and you want Checkmk to automatically accept newly added services without throwing warnings:

1. Go to **Setup** → **Services** → **Periodic service discovery**.
2. Edit the rule **Periodic service discovery**.
3. Under **Severity of unmonitored services**, change it from `WARNING` to `OK`, or set

3. Under **Severity of unmonitored services**, change it from "WARNING" to "OK", or set **Automatic refresh / discovery** to automatically add new services.

Host
Display
Help
Save & run service discovery

Click **Save & run service discovery** at the top left of the action menu.

Step-by-Step Instructions

1. **Click the Button:** Click **Save & run service discovery** in the top action bar.
2. **Accept the New Interface:** On the discovery results page, look for the section titled **Undecided services (1)** showing `lnx_if: Interface 4`. Click **Fix All** (or **Accept all**).
3. **Activate Changes:** Click the yellow **Pending Changes** button in the top right corner of the Checkmk interface and click **Activate on selected sites.**

Verification

Navigate back to the main dashboard or search for **HQ-SRV** under **Services**. The **Check_MK Discovery** service will transition back to an **OK** state.

according to the requirements the domain name is purple.raccoon.lab
so dc01 is the primary domain controller, is not it?

No, based on standard lab architecture and requirements, `dc01` **(or** `HQ-DC01` **) is the Active Directory Domain Controller**, while `hq-srv` is typically a Linux application/infrastructure server (running services like Graylog, Chrony, and Checkmk).

To confirm whether `dc01` is actively serving as the primary Domain Controller for `purple.raccoon.lab`:

Step 1: Verify Active Directory & DNS Roles on `dc01`

Run the following command in PowerShell on `dc01` (or remotely via management tools):

PowerShell

```
Get-ADDomainController -Filter * | Select-Name, Domain, Forest, OperationMasterR
```

Expected Result:

- `Domain : purple.raccoon.lab`
- `OperationMasterRoles` : Lists FSMO roles (e.g., `PDC` , `RIDMaster` , `InfrastructureMaster`), confirming `dc01` holds the Primary Domain Controller (PDC) Emulator role.

Step 2: Query Domain Controllers via DNS

From `hq-srv` (or any host on the network), run `nslookup` to query the DNS SRV records for the domain controller in `purple.raccoon.lab` :

Bash

```
nslookup -type=SRV _ldap._tcp.dc._msdcs.purple.raccoon.lab
```

Expected Result:
The answer section will list the hostname of the Active Directory Domain Controller (e.g., `dc01.purple.raccoon.lab` or `hq-dc01.purple.raccoon.lab`).

Step 3: Test AD Authentication / Port Connectivity

From `hq-srv`, verify that standard Active Directory ports (389 for LDAP, 88 for Kerberos, 53 for DNS) are reachable on `dc01`:

Bash

```
nc -zv dc01.purple.raccoon.lab 53 88 389
```

how to check if static ip is set?

root@hv01:~# uname -a
Linux hv01 6.8.12-9-pve #1 SMP PREEMPT_DYNAMI(

To check if a static IP is configured on Proxmox VE (Debian 12 / Bookworm), inspect both the network configuration files and the active runtime interface status.

Method 1: Check Network Configuration File (Primary Method)

Proxmox VE uses `/etc/network/interfaces` to configure network adapters and bridges (such as `vmbr0`).

Run:

Bash

```
cat /etc/network/interfaces
```

How to verify:

Static IP: Look for `iface <interface_name> inet static` under your primary interface or bridge (`vmbr0`). You will see explicit `address` and `gateway` lines:

Plaintext

```
auto vmbr0
iface vmbr0 inet static
    address 10.101.50.X/24
    gateway 10.101.50.254
    bridge-ports enp0s31f1
    bridge-stp off
    bridge-fd 0
```

DHCP: If it is set via DHCP instead, you will see `iface <interface_name> inet dhcp` .

Method 2: Check Active Runtime Lease/Dynamic Status

To verify whether the currently active IP on an interface was assigned dynamically via DHCP or statically assigned:

Run:

Bash

```
ip -4 addr show
```

How to verify:

DHCP assigned: The interface output will include the flag `dynamic` next to the IP address:

Plaintext

```
inet 10.101.50.10/24 brd 10.101.50.255 scope global dynamic vmbr0
```

Static assigned: The flag `dynamic` will be missing:

Plaintext

```
inet 10.101.50.10/24 brd 10.101.50.255 scope global vmbr0
```

Method 3: Check NetworkManager (If installed)

If NetworkManager is used on the host instead of `ifupdown2`:

Bash

```
nmcli connection show
```

Or check a specific interface:

Bash

```
nmcli dev show <interface_name> | grep IP4.ADDRESS
```

after installation dc01 can ping DC01 - HQ-APC but can not HQ-APC - DC01 also can connect via rdp HQ-APC - DC01 but if firewall on DC01 is totally turn off everything works how to keep firewall turn on and make everything works?

When HQ-APC cannot ping DC01 or connect via RDP, but turning off the firewall on DC01 fixes both, it means **DC01 is blocking inbound ICMP (Ping) and RDP (Port 3389) traffic.**

To keep the Windows Defender Firewall enabled on DC01 while allowing ping and RDP, enable the built-in inbound firewall rules on **DC01.**

Step 1: Allow ICMP (Ping) and RDP via PowerShell on DC01

Run PowerShell as Administrator on **DC01**:

PowerShell

```
# 1. Enable ICMPv4 (Ping) Inbound
Enable-NetFirewallRule -Name "FPS-ICMP4-ERQ-In"

# 2. Enable Remote Desktop (RDP) Inbound Rules
Enable-NetFirewallRule -DisplayGroup "Remote Desktop"
```

Step 2: Allow ICMP (Ping) via GUI (Alternative Method)

If you prefer using the graphical interface on **DC01**:

1. Press `Win + R`, type `wf.msc`, and press **Enter** to open *Windows Defender Firewall with Advanced Security*.
2. Click **Inbound Rules** in the left panel.
3. Locate **File and Printer Sharing (Echo Request - ICMPv4-In)**.
4. Right-click the rule and select **Enable Rule**.

Step 3: Ensure Network Profile is Set to Domain / Private

If DC01's network connection is misidentified as "Public", Windows Firewall blocks inbound connections by default.

Verify the active network profile in PowerShell:

PowerShell

```
Get-NetConnectionProfile
```

Verification:

The `NetworkCategory` should output `DomainAuthenticated` or `Private`.

If it says `Public`, change it using:

PowerShell

```
Set-NetConnectionProfile -InterfaceAlias "Ethernet" -NetworkCategory Private
```

Step 4: Test Connectivity from HQ-APC

From **HQ-APC**, test both ping and RDP to DC01:

DOS

```
:: Test Ping
ping DC01

:: Test RDP Port Connectivity
powershell -Command "Test-NetConnection -ComputerName DC01 -Port 3389"
```

How to verify success:

- `ping DC01` returns active replies.
- `TcpTestSucceeded` returns `True` for port `3389`.

```
Test-NetConnection -ComputerName 10.101.50.11 -Port 3389
WARNING: TCP connect to (10.101.50.11 : 3389) failed
```

`PingSucceeded : True` confirms network routing and ICMP are working, but `TcpTestSucceeded : False` means **port 3389 (RDP) is blocked or the Remote Desktop service is disabled on `10.101.50.11` (DC01).**

Execute the following steps directly on **DC01** to fix it:

Step 1: Enable Remote Desktop Service

Open PowerShell as Administrator on **DC01** and run:

PowerShell

```
# Enable Remote Desktop via Registry
Set-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server'

# Set service startup to Automatic and start it
Set-Service -Name "TermService" -StartupType Automatic
Start-Service -Name "TermService"
```

Step 2: Open RDP Port in Windows Firewall

Enable the necessary inbound firewall rules for Remote Desktop:

PowerShell

```
# Enable built-in Remote Desktop firewall rules across all network profiles
```

```
Enable-NetFirewallRule -DisplayGroup "Remote Desktop"
```

If using a custom rule specifically targeting port 3389:

PowerShell

```
New-NetFirewallRule -DisplayName "Allow RDP Port 3389" -Direction Inbound -Local
```

Step 3: Verify RDP Port Status on DC01

Verify locally on **DC01** that the system is actively listening on port 3389:

PowerShell

```
Get-NetTCPConnection -LocalPort 3389 -State Listen
```

Step 4: Re-test from HQ-APC (`10.101.50.100`)

Run the test again from **HQ-APC**:

PowerShell

```
Test-NetConnection -ComputerName 10.101.50.11 -Port 3389
```

How to verify success:

- `TcpTestSucceeded : True`

[Window Title]
Remote Desktop Connection

[Content]

This error occurs because **Network Level Authentication (NLA)** is enforcing strict CredSSP or Kerberos/NTLM authentication checks that **HQ-APC** cannot satisfy—either due to a missing CredSSP update on one host, unjoined domain status, or CredSSP Encryption Oracle Remediation settings blocking the handshake to `10.101.50.11` (DC01).

Solution 1: Temporarily Disable NLA on DC01 (`10.101.50.11`)

The fastest way to bypass this error and allow RDP connections to DC01 is to disable Network Level Authentication requirement on DC01.

Run the following commands in **PowerShell as Administrator on DC01**:

PowerShell

```
# Disable NLA requirement for RDP connections
(Get-WmiObject -class "Win32_TSGeneralSetting" -Namespace "root\cimv2\terminalse

# Verify user authentication setting (0 = NLA Disabled, 1 = NLA Enabled)
Get-WmiObject -class "Win32_TSGeneralSetting" -Namespace "root\cimv2\terminalser
```

Solution 2: Allow Vulnerable CredSSP Fallback on Client (HQ-APC)

If you cannot access DC01's local console to disable NLA, configure **HQ-APC** to allow connections to servers with unpatched or mismatched CredSSP configurations:

Run this command in **PowerShell as Administrator on HQ-APC**:

PowerShell

```
# Set CredSSP Encryption Oracle Remediation to "Vulnerable" (Value 2)
New-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies
```

Solution 3: Connect Using FQDN Instead of IP Address

If NTLM blocking or domain authentication enforcement is active on DC01:

1. Connect via RDP using the **Fully Qualified Domain Name (FQDN)** instead of the IP address (`10.101.50.11`):

DOS

```
mstsc /v:dc01.purple.raccoon.lab
```

2. Enter your credentials as `PURPLE\Username` or `username@purple.raccoon.lab` .

Verification

Retry the connection from **HQ-APC**:

DOS

```
mstsc /v:10.101.50.11
```

How to verify success: The login prompt appears directly without throwing the *"The function requested is not supported"* CredSSP error.

[Window Title]
Remote Desktop Connection

[Content]

Connecting by hostname `dc01` hits the same underlying CredSSP/NTLM restriction as connecting by IP address. Since NTLM traffic or CredSSP encryption oracle enforcement is blocking the credentials handshake, apply one of the fixes below to establish the connection.

Solution 1: Disable NLA Requirement on `dc01`

Execute the following commands in **PowerShell as Administrator on** `dc01` :

PowerShell

```
# Disable Network Level Authentication requirement
(Get-WmiObject -class "Win32_TSGeneralSetting" -Namespace "root\cimv2\terminalse
```

Verify:

PowerShell

```
Get-WmiObject -class "Win32_TSGeneralSetting" -Namespace "root\cimv2\terminalser
```

Output should show `0` *(Disabled).*

Solution 2: Enable CredSSP Fallback on Client (HQ-APC)

If you are initiating the RDP session from **HQ-APC**, configure the registry on HQ-APC to allow connections to servers with unpatched or strict CredSSP settings:

Run in **PowerShell as Administrator on HQ-APC**:

PowerShell

```
# Create key if missing and set AllowEncryptionOracle to 2 (Vulnerable/Allow)
New-Item -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System'
New-Item -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\
Set-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies
```

Solution 3: Use FQDN + Domain Credentials

If NTLM blocking policy is enforced on `dc01`, connecting with short name `dc01` causes NTLM fallback, which gets rejected. Connecting via full Active Directory FQDN forces Kerberos authentication instead:

1. Launch RDP with the FQDN:

DOS

```
mstsc /v:dc01.purple.raccoon.lab
```

2. When prompted for credentials, explicitly enter the domain format:
   - **User:** `PURPLE\Administrator` (or `administrator@purple.raccoon.lab`)
   - **Password:** *(Your Domain Admin password)*

Verification

Relaunch your RDP session:

DOS

```
mstsc /v:dc01.purple.raccoon.lab
```

How to verify success: The Windows login screen or desktop renders without the CredSSP error pop-up.

Set-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\CredSSP\Parameters' -Name 'AllowEncryptionOracle' -Value 2 -Type DWord -Force

The error occurs because the `CredSSP\Parameters` registry key path does not exist yet on **HQ-APC**.

You must create the missing registry keys first using `New-Item` before setting the property, or combine both in a single step using `-Force`.

Solution: Create the Path and Set the Property

Run this script in **PowerShell as Administrator on HQ-APC**:

PowerShell

```
# Create the missing registry keys and set AllowEncryptionOracle in one step
New-Item -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\
Set-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies
```

Step-by-Step Verification

1. **Verify the registry key was created:**

PowerShell

```
Get-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies
```

Expected Output: `AllowEncryptionOracle : 2`

. **Test RDP Connection Again:**

DOS

```
mstsc /v:10.101.50.11
```

How to verify success: The CredSSP authentication error will no longer appear, and the login window or desktop session will open.

New-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters' -Name 'DisabledComponents' -Value 0xFFFFFFFF -PropertyType DWord -Force

The command you ran is designed to **completely disable IPv6** on all network adapters in Windows by setting the `DisabledComponents` registry value to `0xFFFFFFFF` (4294967295).

Command Breakdown

- **Path:** `HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters`
  - Targets the core kernel driver settings for IPv6.
- **Name:** `DisabledComponents`
  - Controls IPv6 functionality levels across the operating system.
- **Value:** `0xFFFFFFFF`

    - Fully disables IPv6 on all network interfaces, tunnel interfaces, and loopback.
- **PropertyType / Force:** `DWord` creates or overwrites the 32-bit integer key forcefully.

Important Considerations for Active Directory / Windows Server

1. **Reboot Required:** Changes to `DisabledComponents` take effect only after a full system reboot.
2. **Impact on Active Directory:** Microsoft strongly advises against completely disabling IPv6 on Active Directory Domain Controllers (`dc01`). Disabling IPv6 can break core AD services, internal DNS resolution, and group policy processing.
3. **Recommended Alternative:** Rather than disabling IPv6 completely (`0xFFFFFFFF`), Microsoft recommends preferring IPv4 over IPv6 while keeping IPv6 enabled (`0x20`).

Verification & Undo Options

1. Verify the key was created correctly:

PowerShell

```
Get-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameter
```

2. Prefer IPv4 over IPv6 instead (Microsoft Best Practice):

PowerShell

```
Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameter
```

3. Re-enable default IPv6 behavior:

PowerShell

```
Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameter
```

windows server powershell how to remove extra dns server for an appropriate interface?

To remove an extra DNS server from a specific network interface in Windows Server using PowerShell, you can update the IPv4 DNS server addresses list directly using the `Set-DnsClientServerAddress` cmdlet.

Step 1: Identify the Interface Alias or Index

List all network adapters on the server to find the **InterfaceAlias** (e.g., `"Ethernet"` or `"Lab"`) or **InterfaceIndex**:

PowerShell

```
Get-NetAdapter | Select-Name, InterfaceAlias, InterfaceIndex, Status
```

Step 2: Check Current DNS Configuration

Verify the current list of DNS servers configured on your target interface:

PowerShell

```
Get-DnsClientServerAddress -InterfaceAlias "Ethernet" -AddressFamily IPv4
```

Step 3: Remove the Extra DNS Server

PowerShell sets DNS addresses as an ordered array. To keep only the primary DNS server (e.g., `10.101.50.11`) and remove any secondary/extra servers, pass only the desired IP address(es) to `-ServerAddresses`:

PowerShell

```
# Set a single primary DNS server (removes all others)
Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses ("10.101.
```

If you have multiple valid DNS servers and only want to remove one specific extra IP (e.g., remove `8.8.8.8` while keeping `10.101.50.11` and `10.101.50.12`), retrieve the list dynamically, filter out the unwanted IP, and apply the update:

PowerShell

```
$interface = "Ethernet"
$ipToRemove = "8.8.8.8"

# Get current DNS servers and filter out the target IP
$currentDNS = (Get-DnsClientServerAddress -InterfaceAlias $interface -AddressFam
$newDNS = $currentDNS | Where-Object { $_ -ne $ipToRemove }

# Apply updated list
Set-DnsClientServerAddress -InterfaceAlias $interface -ServerAddresses $newDNS
```

Step 4: Reset to DHCP (Optional)

If you want to completely clear static DNS entries and let the interface pull DNS servers automatically via DHCP:

PowerShell

```
Reset-DnsClientServerAddress -InterfaceAlias "Ethernet"
```

Verification

Confirm that only the required DNS server(s) remain on the interface:

PowerShell

```
Get-DnsClientServerAddress -InterfaceAlias "Ethernet" -AddressFamily IPv4
```

ubuntu 24 netplan provide yaml with dns

Here is a standard **Netplan YAML configuration** for Ubuntu 24.04 LTS (`/etc/netplan/01-netcfg.yaml`) with static IP addressing and explicitly defined primary and secondary DNS servers.

Netplan YAML Configuration

YAML

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: false
      dhcp6: false
      addresses:
```

```
    - 10.101.50.40/24
  routes:
    - to: default
      via: 10.101.50.254
  nameservers:
    addresses:
      - 10.101.50.11
      - 1.1.1.1
    search:
      - purple.raccoon.lab
```

Step-by-Step Instructions

① Edit Netplan Configuration File

Requires root permissions

Open or create your netplan configuration file in `/etc/netplan/`:

Bash

```
sudo nano /etc/netplan/01-netcfg.yaml
```

(Note: Replace `eth0` with your actual network interface name, which you can find using `ip link`.)

② Set File Permissions

Security Best Practice

Ensure strict file permissions so only root can read or modify the file:

Bash

```
sudo chmod 600 /etc/netplan/01-netcfg.yaml
```

③ Test and Apply Configuration

Safe rollback enabled

Test the syntax and apply the configuration safely:

Bash

```
sudo netplan try
```

If connection is maintained, press **ENTER** to confirm. Alternatively, apply directly:

Bash

```
sudo netplan apply
```

Verification

Check that the DNS servers are correctly recognized by `systemd-resolved`:

Bash

```
resolvectl status eth0
```

How to verify success: Look for the `DNS Servers:` section in the output. It should list `10.101.50.11` and `1.1.1.1`, and the `DNS Domain:` should reflect `purple.raccoon.lab`.

ubuntu 24 restart network service

On Ubuntu 24.04 LTS, networking is managed by **Netplan** and **systemd-networkd** (or **NetworkManager** on desktop installs). The old `systemctl restart networking` command is deprecated.

Method 1: Netplan (Recommended for Server Install)

To test and apply changes made to your `/etc/netplan/*.yaml` files:

Bash

```
sudo netplan apply
```

If you want to test configuration changes safely with an automatic rollback if you lose connection:

Bash

```
sudo netplan try
```

Method 2: Restart `systemd-networkd` Backend

If you are using the default server backend (`renderer: networkd`), restart the network daemon directly:

Bash

```
sudo systemctl restart systemd-networkd
```

Method 3: Restart NetworkManager (Desktop Install)

Method 3: Restart NetworkManager (Desktop Install)

If your Netplan configuration uses `renderer: NetworkManager` (common on desktop installations):

Bash

```
sudo systemctl restart NetworkManager
```

Step-by-Step Interface Reset

To bounce a specific network interface (e.g., `eth0` or `ens33`) without restarting all system networking services:

Bash

```
sudo ip link set dev eth0 down
sudo ip link set dev eth0 up
```

Verification

Verify that your interfaces and IP assignments are active:

Bash

```
ip addr show
```

Check DNS status and active name servers:

Bash

```
resolvectl status
```

```
root@hv01:~# qm list
      VMID NAME                 STATUS     MEM(MB)    BOOTDISK(GB)
PID
       100 DC01                 running    4096              60.00
```

Your Proxmox host (`hv01`) firewall service (`pve-firewall`) is **disabled** at the datacenter/host level, and both `iptables` and `nftables` have completely open default policies (`ACCEPT`) with no active rule chains.

Even though the VM configurations explicitly include `firewall=1` on their network interfaces (`net0`), the host is **not dropping any traffic** at the hypervisor layer.

If traffic between your VMs (or between `HQ-APC` and `DC01`) is being dropped or failing, the block is occurring **inside the guest operating systems themselves** (e.g., Windows Defender Firewall inside `DC01` / `DC02` / `FS01`) or at an upstream network gateway router—not on Proxmox host `hv01` .

Key Findings from Your Output

- **VM Firewall Flags (`firewall=1`):** Enabled on `VM 100 (DC01)` , `VM 101 (DC02)` , and `VM 102 (FS01)` . However, Proxmox bridge filtering rules for tap interfaces are created only when `pve-firewall` is active.
- **Host Firewall Status (`pve-firewall status`):** `Status: disabled/running` . The daemon process is running, but firewall processing is globally disabled in Proxmox.
- **Kernel Packet Filtering (`iptables` / `nft`):**
  - `iptables` shows standard default ACCEPT chains without any Proxmox bridge chains (`tap100i0` , `PVEFW-*` , etc.).
  - `nft list ruleset` returned blank output.

Proxmox Management Commands

If you ever need to enable or verify Proxmox hypervisor-level filtering, use the following commands:

① Enable Proxmox Firewall Globally

Host Level

Enable Proxmox firewall enforcement on `hv01`:

Bash

```
pve-firewall start
```

② Verify Active Status

Check Service

Confirm that status updates to enabled:

Bash

```
pve-firewall status
```

Verification: Output will display `Status: enabled/running`.

③ Inspect Active Tap Filtering

Kernel Verification

Check that Proxmox generated the interface chains:

Bash

```
iptables-save | grep -i tap
```

Verification: Shows filtering rules tied to `tap100i0`, `tap101i0`, and `tap102i0`.

how to create site-link between HQ and CP

To create a Site Link between two Active Directory sites (e.g., **HQ** and **CP**) in `purple.raccoon.lab`, you define the link inside **Active Directory Sites and Services**. This instructs Active Directory Domain Controllers on how to route replication traffic between the two subnets.

Method 1: Active Directory Sites and Services (GUI)

1. Open **Active Directory Sites and Services** on `DC01` (`dssite.msc`).
2. Expand the **Sites** folder in the left navigation pane.
3. Expand **Inter-Site Transports** and select **IP**.
4. Right-click **IP** and select **New Site Link...**.
5. In the **Name** field, enter a name (e.g., `DEFAULTIPSITELINK-HQ-CP` or `LINK-HQ-CP`).
6. Under **Sites in this site link**, select **HQ** and **CP**, then click **Add** to move both to the right column.
7. Click **OK**.
8. *(Optional)* Right-click the newly created site link → **Properties** to adjust:
   - **Cost:** Determines link preference (lower cost = higher priority, default is `100`).
   - **Replicate every:** Sets replication frequency in minutes (default is `180` minutes).

Method 2: PowerShell (CLI)

Run PowerShell as Administrator on your Domain Controller (`DC01`):

PowerShell

```
# Create the site link between HQ and CP
New-ADReplicationSiteLink -Name "LINK-HQ-CP" `
                          -SitesIncluded "HQ", "CP" `
                          -Cost 100 `
                          -ReplicationFrequencyInMinutes 15 `
                          -InterSiteTransportType IP
```

Step 3: Verify Site Link Creation

Verify that the site link exists and includes both sites:

PowerShell

```
Get-ADReplicationSiteLink -Filter "Name -like '*HQ-CP*'" | Select-Name, Cost, Re
```

How to verify success:

- The output lists `HQ` and `CP` under `SitesIncluded`.
- Active Directory Domain Controllers assigned to the `HQ` site will now use this link to replicate directory data with Domain Controllers in the `CP` site.

lr01 is a linux how to add it to the dns (forward, reverse zones)?

To add a static DNS record for `lr01` (both forward and reverse lookup zones) on your Windows Server Domain Controller (`DC01`) using PowerShell, follow these steps.

Step 1: Add Forward Lookup Record (A Record)

Step 1: Add Forward Lookup Record (A Record)

Create an `A` record mapping `lr01.purple.raccoon.lab` to its IP address (assuming `10.101.50.254` based on your lab gateway):

PowerShell

```
Add-DnsServerResourceRecordA -Name "lr01" -ZoneName "purple.raccoon.lab" -IPv4Ad
```

Step 2: Ensure Reverse Lookup Zone Exists

Check if the reverse lookup zone for your subnet (`50.101.10.in-addr.arpa`) exists on `DC01` :

PowerShell

```
Get-DnsServerZone -Name "50.101.10.in-addr.arpa"
```

If it does **not** exist, create it first:

PowerShell

```
Add-DnsServerPrimaryZone -NetworkId "10.101.50.0/24" -ReplicationScope "Domain"
```

Step 3: Add Reverse Lookup Record (PTR Record)

Create the `PTR` record mapping the IP address back to the host FQDN:

PowerShell

```
Add-DnsServerResourceRecordPTR -Name "254" -ZoneName "50.101.10.in-addr.arpa" -P
```

```
Add-DnsServerResourceRecordPtr -Name "254" -ZoneName "50.101.10.in-addr.arpa" -F
```

Verification

Test both forward and reverse resolution from any host in the domain:

PowerShell

```
# Test Forward Resolution
Resolve-DnsName -Name "lr01.purple.raccoon.lab"

# Test Reverse Resolution
Resolve-DnsName -Name "10.101.50.254"
```

How to verify success:

- `Resolve-DnsName lr01.purple.raccoon.lab` returns `10.101.50.254`.
- `Resolve-DnsName 10.101.50.254` returns `lr01.purple.raccoon.lab`.

What is InstallDNS default value? Will it be installed?

```
Install-ADDSDomainController `
-DomainName "purple.raccoon.lab" `
```

The default value for the `-InstallDNS` parameter in `Install-ADDSDomainController` is `$true` (enabled).

In your specific command, **DNS will be installed.**

Details

- **Explicit switch provided:** Because you included `-InstallDNS` in your command without specifying `$false` (e.g., `-InstallDNS:$false`), PowerShell evaluates this switch parameter as `$true`.
- **Default cmdlet behavior:** Even if you omit `-InstallDNS` entirely when promoting a Domain Controller with `Install-ADDSDomainController`, the cmdlet defaults to installing and configuring the DNS Server role automatically, unless `-InstallDNS:$false` is explicitly passed.

what ports should be opened on a router and how to chech if cp-dc04 can replicate with hq-dc01 throught router?

PS C:\Users\Administrator.PURPLE> repadmin /syn

The error `1722 (0x6ba): The RPC server is unavailable` during Active Directory replication indicates that traffic is being blocked by a router/firewall or that RPC dynamic ports cannot be allocated.

1. Required Firewall Ports for Active Directory Replication

To allow full AD replication between `CP-DC04` and `HQ-DC01` across a router or firewall, open the following inbound ports in both directions:

Service / Protocol	Port(s)	Transport
RPC Endpoint Mapper	`135`	TCP
RPC Dynamic Ports	`49152-65535` *(default)* or `50000` *(fixed)*	TCP
Kerberos	`88`	TCP / UDP

DNS	53	TCP / UDP
LDAP	389	TCP / UDP
LDAP Secure (LDAPS)	636	TCP
Global Catalog	3268 / 3269	TCP
SMB (SYSVOL / Netlogon)	445	TCP
W32Time (Time Sync)	123	UDP

Best Practice Tip: Standard high RPC dynamic ports (`49152-65535`) are wide. You can restrict Active Directory RPC traffic to a fixed single port (e.g., TCP `50000`) on both DCs by setting a static NTDS RPC port in the registry key: `HKLM\SYSTEM\CurrentControlSet\Services\NTDS\Parameters` → REG_DWORD `TCP/IP Port = 50000` .

2. How to Test Connectivity from `CP-DC04` to `HQ-DC01`

Run these tests directly in PowerShell from `CP-DC04` :

Test Essential Fixed Ports

PowerShell

```powershell
$ports = 53, 88, 135, 389, 445, 3268
foreach ($port in $ports) {
    Test-NetConnection -ComputerName "HQ-DC01" -Port $port |
        Select-Object ComputerName, RemotePort, TcpTestSucceeded
}
```

Test RPC Endpoint Mapper Specifically

Verify if the RPC service is reachable and responding:

PowerShell

```
nltest /sc_query:purple.raccoon.lab
```

3. How to Check and Force Replication Status

Once ports are open on the router, use the following native diagnostic tools:

Check Full Replication Summary

Run on either DC to view partner connectivity status across all sites:

PowerShell

```
repadmin /replsummary
```

Test Specific Inbound Link (CP-DC04 ↔ HQ-DC01)

Force a direct replication sync of the domain partition between `CP-DC04` and `HQ-DC01`:

PowerShell

```
# Run on CP-DC04 to force pull from HQ-DC01
repadmin /syncall /A /e /P HQ-DC01
```

Check Active Directory Replication Status

Verify if any replication errors exist across the domain:

PowerShell

```
Get-ADReplicationPartnerMetadata -Target "CP-DC04" | Select-Object Server, Partn
```

How to verify success:

- `repadmin /replsummary` shows `0` errors for both `CP-DC04` and `HQ-DC01`.
- `LastReplicationResult` returns `0` (Success).

how to check if wireguard establish connection?

linux_admin@lr01:~$ sudo wg show
interface: wg1

To determine if a WireGuard connection is successfully established, evaluate two key fields in `wg show`: `latest handshake` and `transfer`.

Looking at your `sudo wg show` output:

- `wg1` **is UP and CONNECTED**:
  - `latest handshake: 1 minute, 14 seconds ago` — Handshakes occur every 2–3 minutes. Anything under ~3 minutes means the tunnel is authenticated and established.
  - `transfer: 715.51 MiB received, 307.83 MiB sent` — Non-zero data received confirms bidirectional traffic.
- `wg0` **is DOWN / NOT CONNECTED**:
  - `latest handshake:` Missing entirely.
  - `transfer: 0 B received, 444 B sent` — Your host is sending packets, but no responses are returning (0 B received).

Step-by-Step Verification Checklist

1. **Check Handshake Status (Layer 3 Tunnel Level)**
   Run `sudo wg show` or explicitly request the timestamp:

   Bash

   ```
   sudo wg show wg0 latest-handshakes
   ```

   - **Connected:** Returns an epoch timestamp or a time less than 3 minutes ago.
   - **Disconnected:** Returns `0` or no handshake line at all.
2. **Test Subnet Reachability (Ping across the tunnel)**
   Ping the remote peer's tunnel IP address (defined in `allowed ips`):

   Bash

   ```
   # Test wg1 tunnel IP
   ping -c 3 10.251.101.22

   # Test wg0 tunnel IP
   ping -c 3 10.250.101.1
   ```

3. **Check WireGuard Routes**
   Verify system routing table entries for the allowed IPs:

   Bash

   ```
   ip route show dev wg1
   ip route show dev wg0
   ```

4. **Troubleshoot `wg0` Connection**
   Since `wg0` is sending traffic (`444 B sent`) but receiving nothing (`0 B`):
   - Verify the public key on the remote peer (`203.0.113.10`) matches `M1Sxx8Msn68uRG5NoLLsTlAqE/FNXZROSQ8DocsuPyw=` .
   - Ensure UDP port `51820` is open on the remote endpoint router/firewall.

- Ensure UDP port `51820` is open on the remote endpoint router/firewall.
- Verify the endpoint IP `203.0.113.10` is reachable from `lr01`.

```
linux_admin@lr01:~$ sudo ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens3             UP             10.101.50.254/24
fe80::f816:3eff:fed8:112e/64
```

Your diagnostic tests narrow the issue with `wg0` down to a specific configuration failure on either `lr01` or the remote peer (`203.0.113.10`).

Diagnostic Findings

- `wg1` **is working properly:** Pings to peer `10.251.101.22` return sub-2ms latency with zero packet loss.
- `wg0` **path is reachable:** Direct ICMP reachability to the WAN endpoint (`203.0.113.10`) via `ens4` succeeds with 0% loss.
- `wg0` **handshakes fail completely:** `latest-handshakes` returns `0`, confirming the UDP handshake sequence never completes.

Root Cause Analysis & Systematic Troubleshooting

Because UDP traffic reaches the remote host's IP address, the failure of WireGuard to complete a handshake is caused by one of three reasons:

1. Public Key Mismatch

WireGuard silently drops incoming packets if the sender's public key isn't explicitly configured in the peer block.

Fix: Verify the public key generated on `lr01` for `wg0` matches the peer configuration on `203.0.113.10`:

Bash

```
sudo wg show wg0 public-key
```

Ensure the public key defined under `[Peer]` on `203.0.113.10` exactly matches this output.

2. Port Mismatch or Firewall Drop on Peer

While ICMP (Ping) is open to `203.0.113.10`, UDP port `51820` (or `51821`) might be blocked by `iptables`, `nftables`, or an upstream cloud security group on the remote peer.

Fix on Remote Peer (`203.0.113.10`): Ensure the UDP listening port is open in the firewall:

Bash

```
# Example for iptables
sudo iptables -A INPUT -p udp --dport 51820 -j ACCEPT
```

3. Misconfigured Endpoint or Port in `/etc/wireguard/wg0.conf`

Check your local `wg0` configuration file on `lr01`:

Bash

```
sudo cat /etc/wireguard/wg0.conf
```

Verify the following directives under `[Peer]`:

- `Endpoint = 203.0.113.10:<PORT>`: Confirm `<PORT>` matches the exact `ListenPort` configured inside the remote peer's WireGuard configuration file (not necessarily `51820` if `wg1` is already using it).
- `AllowedIPs = 10.250.101.0/24, 10.100.101.0/24`: Confirm the remote interface

- `AllowedIPs = 10.250.101.0/24, 10.100.101.0/24` . Confirm the remote interface address (`10.250.101.1/32` or subnet) is included.

Real-Time Handshake Capture Test

Run a packet capture on `ens4` while sending traffic to observe if UDP packets exit `lr01` and receive any response:

Bash

```
sudo tcpdump -ni ens4 host 203.0.113.10 and udp
```

- **If you only see outbound packets (`out`):** The remote peer `203.0.113.10` is dropping UDP traffic on port `51820` due to a firewall rule or public key mismatch.
- **If you see inbound replies (`in`):** `lr01` is dropping the incoming packets due to a mismatched peer public key in `/etc/wireguard/wg0.conf` .

linux_admin@lr01:~$ sudo wg show wg0 public-key
M1Sxx8Msn68uRG5NoLLsTlAqE/FNXZROSQ8DocsuPyw=
linux_admin@lr01:~$ sudo cat /etc/wireguard/wg0.conf
[Interface]

Your local `/etc/wireguard/wg0.conf` configuration on `lr01` is structurally correct.

Given that `wg0` uses `ListenPort = 51821` locally and targets `Endpoint = 203.0.113.10:51820` , the issue is isolated to the remote peer (**CHQ-LR** at `203.0.113.10`) or the network path between them.

Step-by-Step Diagnostic & Fixes

1. **Verify Key Pairing on Remote Peer (CHQ-LR)**

On `203.0.113.10`, inspect the WireGuard configuration file (usually `/etc/wireguard/wg0.conf` or similar):

- Under its `[Interface]` section, confirm `ListenPort = 51820`.
- Under the `[Peer]` entry representing `lr01`, verify that:

Ini, TOML

```
PublicKey = M1Sxx8Msn68uRG5NoLLsTlAqE/FNXZROSQ8DocsuPyw=
AllowedIPs = 10.250.101.15/32
```

- *If the remote public key does not match `lr01`'s key exactly, WireGuard on `CHQ-LR` silently discards all handshake attempts.*

3. **Run Packet Capture on `lr01`**
   Execute `tcpdump` on `lr01` to check if handshake packets are leaving `ens4` and if any response returns:

Bash

```
sudo tcpdump -ni ens4 host 203.0.113.10 and udp port 51820
```

- **Only outbound packets visible:** `203.0.113.10` is either blocking UDP `51820` via firewall (`iptables` / `nftables` /cloud security group) or ignoring the packets due to a public key mismatch.
- **Inbound packets visible:** `lr01` is rejecting the response because the remote public key (`4Z3Nl+...`) defined in `/etc/wireguard/wg0.conf` does not match the key `203.0.113.10` is signing with.

4. **Check Firewall on Remote Peer (CHQ-LR)**
   Ensure UDP port `51820` is explicitly permitted inbound on `203.0.113.10`:

Bash

```
# UFW example
sudo ufw allow 51820/udp

# IPTables example
```

```
sudo iptables -A INPUT -p udp --dport 51820 -j ACCEPT
```

- **Restart WireGuard Interface after Edits**
  If you update `/etc/wireguard/wg0.conf` on either end, reload the interface:

Bash

```
sudo wg-quick down wg0 && sudo wg-quick up wg0
```

Verification

After applying corrections on `203.0.113.10`, run:

Bash

```
sudo wg show wg0 latest-handshakes
```

How to verify success: The output returns a non-zero timestamp for peer `4Z3Nl+...`, and `ping -c 3 10.250.101.1` succeeds with 0% packet loss.

linux_admin@lr01:~$ sudo cat /etc/nftables.conf
#!/usr/sbin/nft -f

flush ruleset

The `/etc/nftables.conf` firewall configuration looks well-structured, but there are a few notable **flaws and operational risks** depending on your exact setup:

Issues & Potential Bugs

1. **WireGuard Multi-Interface Port Binding Issue**

Code snippet

```
iifname "ens4" udp dport 51820 accept
```

- **Problem:** You have two WireGuard interfaces (`wg0` and `wg1`). Unless both peers point to the exact same listening port (51820), one of the WireGuard tunnels will fail to establish its handshake. Standard WireGuard setups typically assign distinct UDP ports for different interfaces (e.g., 51820 for `wg0` and 51821 for `wg1`).
- **Fix:** If both use distinct ports, update the input rule to accept the range or multiple ports:

Code snippet

```
iifname "ens4" udp dport { 51820, 51821 } accept
```

2. **Unclamped TCP MSS Over WireGuard Tunnels**
   - **Problem:** WireGuard adds IP/UDP header overhead (usually setting MTU to 1420 or lower). Standard TCP SYN packets from LAN hosts (`ens3`) carry a maximum segment size (MSS) meant for standard 1500-byte MTUs. This causes TCP connections across `wg0` / `wg1` (e.g., large HTTP transfers, SSH sessions, database connections) to stall or hang when large packets get silently dropped.
   - **Fix:** Add a TCP MSS clamping rule to the `forward` chain or a dedicated `mangle` table:

Code snippet

```
tcp flags syn tcp option maxseg size set rt mtu
```

3. **Checkmk Agent Restricted to LAN Interface (`ens3`)**

Code snippet

```
iifname "ens3" ip saddr 10.101.50.0/24 tcp dport 6556 accept
```

- **Behavior:** If your Checkmk monitoring server resides on `LR02` or behind one of the VPN tunnels (`wg0` / `wg1`), it will be blocked because `iifname "ens3"` restricts checks exclusively to local network traffic arriving directly via `ens3` .

Cleaned & Improved `/etc/nftables.conf`

Code snippet

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {

    chain input {
        type filter hook input priority filter;
        policy drop;

        iifname "lo" accept

        ct state established,related accept

        ip protocol icmp accept

        # SSH from HQ LAN
        iifname "ens3" ip saddr 10.101.50.0/24 tcp dport 22 accept

        # Checkmk Agent from HQ LAN
        iifname "ens3" ip saddr 10.101.50.0/24 tcp dport 6556 accept

        # NTP clients from HQ LAN
        iifname "ens3" ip saddr 10.101.50.0/24 udp dport 123 accept

        # WireGuard listening ports on public WAN interface
        iifname "ens4" udp dport { 51820, 51821 } accept

        # VPN tunnel internal endpoints
        iifname { "wg0", "wg1" } accept
    }
```

```
    chain forward {
        type filter hook forward priority filter;
        policy drop;

        ct state established,related accept

        # Clamp TCP MSS to PMTU for VPN traffic to prevent path MTU issues
        oifname { "wg0", "wg1" } tcp flags syn tcp option maxseg size set rt mtu

        # HQ -> CHQ
        iifname "ens3" oifname "wg0" ip saddr 10.101.50.0/24 ip daddr 10.100.101

        # CHQ -> HQ
        iifname "wg0" oifname "ens3" ip saddr 10.100.101.0/24 ip daddr 10.101.50

        # HQ -> CP
        iifname "ens3" oifname "wg1" ip saddr 10.101.50.0/24 ip daddr 10.101.51.

        # CP -> HQ
        iifname "wg1" oifname "ens3" ip saddr 10.101.51.0/24 ip daddr 10.101.50.

        # LR02 host IP -> HQ LAN
        iifname "wg1" oifname "ens3" ip saddr 10.251.101.22 ip daddr 10.101.50.0

        # HQ -> Repository
        iifname "ens3" oifname "ens4" ip saddr 10.101.50.0/24 ip daddr 203.0.113
    }

    chain output {
        type filter hook output priority filter;
        policy accept;
    }
}

table ip nat {

    chain postrouting {
        type nat hook postrouting priority srcnat;
        policy accept;

        # Masquerade outbound external repository traffic
        iifname "ens3" oifname "ens4" ip saddr 10.101.50.0/24 ip daddr 203.0.113
    }
}
```

To test and reload the updated ruleset without locking yourself out:

Bash

```
sudo nft -c -f /etc/nftables.conf && sudo nft -f /etc/nftables.conf
```

```
linux_admin@lr01:~$ sudo wg show
interface: wg1
  public key:
V3FKIJLUOyhyTL71gfG6vuU1apwHRKpzeE8NNuW(
```

Both WireGuard tunnels (`wg0` and `wg1`) are now fully established with active handshakes (`15 seconds ago` on `wg0` and non-zero bytes received).

Since `wg0` is authenticated but `ping 10.100.101.1` fails, the issue is caused by **firewall rules on** `lr01` , **routing/firewall on CHQ-LR (** `203.0.113.10` **)**, or **missing return routes.**

Step 1: Fix `nftables` Drop Rule on `lr01`

In your earlier `/etc/nftables.conf` , the input rule for WireGuard endpoints was:

Code snippet

```
iifname "ens4" udp dport 51820 accept
```

Because `wg0` is bound to `listening port: 51821` , inbound UDP traffic returning to `lr01` on port 51821 or traffic destined for `wg0` 's local socket might be getting dropped if

`nftables` wasn't updated.

Apply this fix on `lr01`:

Bash

```
# Allow both WireGuard ports inbound
sudo nft add rule inet filter input iifname "ens4" udp dport { 51820, 51821 } ac
```

Step 2: Verify `AllowedIPs` and Routing on `lr01`

Ensure `lr01` actually routes `10.100.101.1` out through `wg0`:

Bash

```
ip route get 10.100.101.1
```

Expected output:
`10.100.101.1 dev wg0 src 10.250.101.15 ...`

If it routes via `ens4` *or another interface, your* `AllowedIPs` *under* `[Peer]` *in* `/etc/wireguard/wg0.conf` *is missing* `10.100.101.0/24` *or* `10.100.101.1/32`.

Step 3: Trace Packet Flow with `tcpdump`

Run `tcpdump` on `lr01` while initiating a ping to `10.100.101.1`:

Bash

```
sudo tcpdump -ni wg0 icmp
```

- **Case A: No packets leaving** `wg0` → `nftables` forward chain is dropping the outbound

packet from `ens3` / `local` to `wg0`.

- **Case B: Outbound ICMP Request visible, no Reply** → The packet reaches CHQ-LR (`203.0.113.10`), but CHQ-LR is either dropping it in its firewall or lacks a return route for `10.101.50.0/24` (or `10.250.101.15`) back through its WireGuard tunnel.

Step 4: Verify CHQ-LR Remote Configuration

On the remote endpoint `CHQ-LR` (`203.0.113.10`):

1. **Check IP Forwarding:**

Bash

```
sysctl net.ipv4.ip_forward
```

(Must be `1`)

2. **Check WireGuard Peer `AllowedIPs` on CHQ-LR:**
Under the `[Peer]` section for `lr01` on CHQ-LR, ensure it explicitly includes your source subnet or IP:

Ini, TOML

```
AllowedIPs = 10.250.101.15/32, 10.101.50.0/24
```

If `10.101.50.0/24` (or `10.250.101.15`) is missing from CHQ-LR's `AllowedIPs`, CHQ-LR will silently drop the ping response.

ubunto linux how to check if possible to esteblish connection with an appropriate ip and port

To check if an IP address and port are reachable from an Ubuntu machine, use one of the following methods depending on the protocol (TCP or UDP) and tools installed.

Method 1: Using `nc` (Netcat) — Recommended

Netcat is the quickest tool for checking both TCP and UDP port reachability without sending full application payloads.

1. Test TCP Port Connection:

Bash

```
nc -zv 10.100.101.1 80
```

- `-z` : Zero-I/O mode (scans without sending data).
- `-v` : Verbose mode (displays success or failure).

Output on success: `Connection to 10.100.101.1 80 port [tcp/http] succeeded!`

2. Test UDP Port Connection:

Bash

```
nc -zuv 10.100.101.1 51820
```

- `-u` : Switches Netcat to UDP mode.

Method 2: Using Native Bash `/dev/tcp` (No extra tools required)

If `nc` or `telnet` are not installed, standard Bash can test TCP sockets natively:

Bash

```
timeout 3 bash -c '</dev/tcp/10.100.101.1/80' && echo "Open" || echo "Closed/Fil
```

- **Verification:** If the command returns `Open` and exit code `0`, the socket connection succeeded.

Method 3: Using `curl` (Web & HTTP/S Ports)

For web servers, APIs, or HTTP-based services:

Bash

```
curl -vtelnet://10.100.101.1:80
```

or simply check the connection timeout:

Bash

```
curl --connect-timeout 3 -I http://10.100.101.1:80
```

Method 4: Using `nmap` (Detailed Network Diagnostics)

For detailed port state analysis (Open, Closed, or Filtered by Firewall):

Bash

```
# Check specific TCP port
nmap -p 80 10.100.101.1

# Check specific UDP port
sudo nmap -sU -p 51820 10.100.101.1
```

Understanding the Results

Status Output	Meaning	Common Cause
Succeeded / Open	Port is open and a service is actively listening.	Normal connection.
Connection Refused	Host reached, but no service is listening on that port.	Service is stopped or bound to local `127.0.0.1` only.
Connection Timed Out	Packets are sent, but no reply returned.	Intermediate firewall (`nftables` / `iptables` /cloud security group) is dropping traffic.

exercise

Create
separate GPOs named GPO-Checkmk-Agent, GPC

Here is your structured deployment plan and execution checklist for deploying Checkmk, NXLog, and Graylog GELF inputs across your Windows environment.

Phase 1: Group Policy Object (GPO) Setup

To ensure software installs immediately during boot rather than hanging on background updates, establish the base GPOs and security filters.

1. GPO Creation & Linking

Create three GPOs in Group Policy Management Console (`gpmc.msc`):

- `GPO-Checkmk-Agent`
- `GPO-NXLog-Agent`
- `GPO-NXLog-Config`

Link all three GPOs to:

1. The **Lab** Organizational Unit (OU).
2. The **Domain Controllers** OU.

2. Security Filtering & Computer Startup Settings

For each of the three GPOs:

1. Under **Security Filtering**, remove `Authenticated Users`.
2. Add the security group `gg_monitoring_computers`.
3. Under the **Delegation** tab, click **Advanced** → Add `Authenticated Users` → Grant **Read** permission only (*Do NOT grant "Apply group policy"*).
4. Edit each GPO and enable the network wait policy:
   - **Path:** `Computer Configuration` → `Policies` → `Administrative Templates` → `System` → `Logon`
   - **Setting:** `Always wait for the network at computer startup and logon` = **Enabled**

Phase 2: Graylog GELF TCP Input Setup (HQ-SRV)

Before deploying NXLog agents, open the input receiver on your Graylog host (`HQ-SRV`).

1. Docker Compose Configuration

Edit `docker-compose.yml` on `HQ-SRV` and update the `graylog` service container ports:

YAML

```yaml
ports:
  - "9000:9000"
```

```
      - "1514:1514/tcp"
      - "${HQ_SRV_IP}:12201:12201/tcp"
```

Validate and apply the stack:

Bash

```
sudo docker compose config --quiet
sudo docker compose up -d graylog
```

2. Graylog Web Interface Setup

1. Log into Graylog (`http://<HQ-SRV-LAN-IP>:9000`).
2. Navigate to **System / Inputs** → Select **GELF TCP** → Click **Launch new input**.
3. Configure the input settings:
   - **Global:** Checked
   - **Title:** `Windows-GELF`
   - **Bind address:** `0.0.0.0`
   - **Port:** `12201`
   - **Null frame delimiter:** Checked (`true`)
   - **TLS:** Disabled
4. Click **Save** and verify the input status shows **Running**.
5. Ensure local host firewalls permit inbound TCP `12201` from the POD LANs to `HQ-SRV` .

Phase 3: Checkmk Agent Deployment & TLS Registration

① **Configure Software Package in GPO**

GPO-Checkmk-Agent

Navigate to Computer Configuration → Policies → Software Settings → Software installation. Right-click → New → Package. Select the Checkmk Agent MSI using its absolute UNC path (e.g., \domain.local\sysvol...\check_mk_agent.msi). Select deployment method: Assigned.

deployment method: Assigned.

② **Deploy to Targets and Verify**

Pilot Systems First

On pilot machines, run gpupdate /force and execute a restart. Verify installation via PowerShell:
Get-Service CheckMkService

③ **Register Agent TLS Interactively**

Manual CLI Step

GPO installation does not perform TLS registration. Run the registration command interactively on each target console (never store credentials in scripts or SYSVOL):

&"C:\Program Files (x86)\checkmk\service\cmk-agent-ctl.exe" register --hostname <TARGET_HOSTNAME> --server <HQ-SRV-IP> --site <SITE_NAME> --user cmkadmin

④ **Checkmk Firewall & Discovery**

Port Setup

Ensure firewalls permit traffic between host and monitoring server:

Phase 4: NXLog Deployment & Central Configuration

1. Install NXLog via GPO

1. Open `GPO-NXLog-Agent`.
2. Path: `Computer Configuration` → `Policies` → `Software Settings` → `Software installation`.
3. Add the NXLog MSI using its full UNC share path (`\\share\path\nxlog-x64.msi`) → Set to **Assigned**.

2. Create Central `nxlog.conf`

Save the file as `nxlog.conf` on your distribution share. *(Ensure* `<HQ-SRV-LAN-IP>` *is replaced with the real IP address of HQ-SRV).*

Ini, TOML

```
# Verify ROOT against the installation path of the MSI from the lab share.
define ROOT C:\Program Files\nxlog
Moduledir %ROOT%\modules
CacheDir %ROOT%\data
Pidfile %ROOT%\data\nxlog.pid
SpoolDir %ROOT%\data
LogFile %ROOT%\data\nxlog.log

<Extension gelf>
    Module xm_gelf
</Extension>

<Input windows_events>
    Module im_msvistalog
    ReadFromLast TRUE
    SavePos TRUE
    Query <QueryList><Query Id="0"><Select Path="Application">*</Select><Select
</Input>

<Output graylog>
    Module om_tcp
    Host <HQ-SRV-LAN-IP>
    Port 12201
    OutputType GELF_TCP
</Output>

<Route windows_to_graylog>
    Path windows_events => graylog
</Route>
```

3. Distribute Configuration via Preference GPO

1. Edit `GPO-NXLog-Config`.
2. **File Distribution:**
   - Path: `Computer Configuration` → `Preferences` → `Windows Settings` → `Files`
   - Action: **Replace**

- Source File: `\\share\path\nxlog.conf`
- Destination File: `C:\Program Files\nxlog\conf\nxlog.conf` *(or* `C:\Program Files (x86)\nxlog\conf\nxlog.conf` *for 32-bit deployments)*
- **Item-Level Targeting:** Configure targeting to verify file existence of `C:\Program Files\nxlog\nxlog.exe` prior to replacement.

3. **Service Management:**
   - Path: `Computer Configuration` → `Preferences` → `Control Panel Settings` → `Services`
   - Service Name: `nxlog`
   - Startup type: **Automatic**
   - Service Action: **Start service** (or **Restart service**)

Phase 5: Verification & Deployment Matrix

Execute the following PowerShell snippet on each target (including Server Core DCs) to test agent state and generate test event telemetry:

PowerShell

```powershell
# 1. Update policy
gpupdate /force

# 2. Check applied GPOs
gpresult /scope computer /r

# 3. Verify installed services
Get-Service -Name *check*,*nxlog*

# 4. Check network connectivity to Graylog GELF port
Test-NetConnection <HQ-SRV-LAN-IP> -Port 12201

# 5. Generate test log event
eventcreate /T INFORMATION /ID 100 /L APPLICATION /SO DS_ComplexLab /D "NXLog GP
```

Deployment Tracking Checklist

Host Name	Target Type	GPOs Applied	Checkmk Agei Version
DC-01	Server Core DC	[ ]	v2.x
DC-02	Server Core DC	[ ]	v2.x
HQ-CA	ADCS Server	[ ]	v2.x
APC-01	Member Server	[ ]	v2.x
APC-02	Member Server	[ ]	v2.x
APC-03	Member Server	[ ]	v2.x
SRV-01	Member Server	[ ]	v2.x
SRV-02	Member Server	[ ]	v2.x
SRV-03	Member Server	[ ]	v2.x

Verification Steps in Graylog

1. Log into **Graylog** → Go to **Search**.
2. Run the search query: `source: "DS_ComplexLab"` or `message: "NXLog GPO test"`.
3. Confirm that the log output populates the `Application` channel metadata, correct Windows hostname, and Event ID `100`.
4. Validate that existing Linux Syslog input on TCP port `1514` remains active and continues to ingest logs simultaneously.

15. PKI
Exercise 14 · Configure the POD certification authority
Configure hq-ca01
as an AD-integrated subordinate CA of the classrc

Here is your structured execution guide for configuring the AD-integrated Subordinate CA, HTTP AIA/CDP distribution endpoints, custom certificate templates, and automated domain-wide PKI enrollment.

Step 1: DNS and PKI HTTP Publication Setup (HQ-SRV)

Set up the `pki.local` DNS infrastructure and the lightweight Nginx container to serve CRLs and Authority Information Access (AIA) files over HTTP on TCP port 80.

1. Configure DNS Zone in Active Directory

Execute in PowerShell on `hq-ca01` (or any Domain Controller):

PowerShell

```
# Create AD-integrated Forward Lookup Zone replicated to all DNS servers in the
Add-DnsServerPrimaryZone -Name "pki.local" -ReplicationScope "Forest"

# Add apex A record pointing to HQ-SRV (.40)
Add-DnsServerResourceRecordA -ZoneName "pki.local" -Name "@" -IPv4Address "<HQ-S
```

2. Deploy Nginx Web Container on HQ-SRV

Run the following on `HQ-SRV` to initialize directories and start the Docker container:

Bash

```
sudo mkdir -p /opt/pki-web /opt/www/pki
sudo chmod 755 /opt/www /opt/www/pki
```

```
cd /opt/pki-web

# Create .env file (Replace <Repository> and <HQ-SRV-LAN-IP>)
cat <<EOF | sudo tee .env
REGISTRY=<Repository>:5000
NGINX_TAG=<trainer-provided-tag>
HQ_SRV_IP=<HQ-SRV-LAN-IP>
EOF

# Create compose.yaml
cat <<'EOF' | sudo tee compose.yaml
services:
  nginx:
    image: "${REGISTRY}/library/nginx:${NGINX_TAG}"
    restart: unless-stopped
    ports:
      - "${HQ_SRV_IP}:80:80"
    volumes:
      - /opt/www:/usr/share/nginx/html:ro
EOF

# Validate and start container
sudo docker compose config --quiet
sudo docker compose pull
sudo docker compose up -d
sudo docker compose ps
```

Step 2: Download and Publish Root CA Files

Retrieve the classroom Root CA certificate and CRL, publish them into Active Directory, and expose them via your Nginx web service.

1. Download & AD Publication (On `hq-ca01`)

Run in an elevated Command Prompt or PowerShell:

DOS

```
:: Download Root CA artifacts from CentralHQ Web server
curl.exe -o RootCA.crt http://<CentralHQ-Web-server>/pki/RootCA.crt
```

```
curl.exe -o RootCA.crl http://<CentralHQ-Web-server>/pki/RootCA.crl

:: Publish Root CA Certificate and CRL into Active Directory Forest
certutil -f -dspublish RootCA.crt RootCA
certutil -f -dspublish RootCA.crl
certutil -addstore -f Root RootCA.crt
```

2. Copy Root Artifacts to HTTP Publication Directory

Copy `RootCA.crt` and `RootCA.crl` to `/opt/www/pki/` on `HQ-SRV`. Ensure permissions are set properly:

Bash

```
sudo chmod 644 /opt/www/pki/RootCA.crt /opt/www/pki/RootCA.crl
```

Verify reachability from a workstation or client (`APC`):

- `[http://pki.local/pki/RootCA.crt](http://pki.local/pki/RootCA.crt)`
- `[http://pki.local/pki/RootCA.crl](http://pki.local/pki/RootCA.crl)`

Step 3: Install & Configure Enterprise Subordinate CA

① **Install AD CS Role**

Server Manager or PowerShell

On hq-ca01, install Active Directory Certificate Services:

Install-WindowsFeature ADCS-Cert-Authority -IncludeManagementTools

② **Configure Subordinate CA**

AD CS Configuration Wizard

Configure the role with the following specifications:

③ **Sign Request & Install Certificate**

③ **Sign Request & Install Certificate**

Trainer Interaction

Send the .req file to the trainer. Once returned (e.g., as SubCA.crt):

Open Certification Authority (certsrv.msc) ⟶ Right-click CA Name ⟶ All Tasks ⟶ Install CA Certificate ⟶ Select SubCA.crt ⟶ Start Service.

④ **Configure Extensions (AIA & CDP)**

CA Properties

Right-click CA Name ⟶ Properties ⟶ Extensions tab:

1. CRL Distribution Point (CDP):
Add URL: http://pki.local/pki/

⑤ **Publish SubCA Files to HTTP Server**

Finalize Distribution

Publish a new CRL via certutil -crl. Copy the newly generated Subordinate CA certificate and CRL file(s) from C:\Windows\System32\CertSrv\CertEnroll</code> to HQ-SRV:/opt/www/pki/.

Step 4: Certificate Templates Setup

Open Certificate Templates Console (`certtmpl.msc`) on `hq-ca01` .

1. Custom Member Computer Template (`POD-Computer`)

1. Duplicate **Computer** template ⟶ Name: `POD-Computer` .
2. **Cryptography:** Provider Category: `Key Storage Provider (KSP)` , Algorithm: `RSA` , Min key size: `2048` , Hash: `SHA256` .
3. **Request Handling:** Ensure *Allow private key to be exported* is **unchecked**.
4. **Subject Name:** Select *Build from this Active Directory information* ⟶ Subject name format: *Fully distinguished name*, Include *DNS name* in SAN.
5. **Security:** Add `Domain Computers` ⟶ Allow: **Read**, **Enroll**, **Autoenroll**.

2. Custom Domain Controller Template (`POD-DomainController`)

1. Duplicate **Kerberos Authentication** template → Name: `POD-DomainController` .
2. **Extensions:** Ensure **KDC Authentication**, **Server Authentication**, and **Client Authentication** remain listed in Enhanced Key Usage (EKU).
3. **Subject Name:** Select *Build from this Active Directory information*.
4. **Security:** Add `Domain Controllers` → Allow: **Read**, **Enroll**, **Autoenroll**.

3. Issue Templates

In `certsrv.msc` : Right-click **Certificate Templates** → **New** → **Certificate Template to Issue** → Select `POD-Computer` and `POD-DomainController` .

Step 5: Group Policy for Auto-Enrollment

Create and link `GPO-Certificate-Autoenrollment` to **Lab OU** and **Domain Controllers OU**.

1. **Auto-Enrollment Policy Configuration:**
   - **Path:** `Computer Configuration` → `Policies` → `Windows Settings` → `Security Settings` → `Public Key Policies`
   - **Setting:** `Certificate Services Client - Auto-Enrollment`
   - **Configuration:** Set to **Enabled**.
   - Check options:
     - *Renew expired certificates, update pending certificates, and remove revoked certificates*
     - *Update certificates that use certificate templates*
2. **Trusted Root Import:**
   - **Path:** `Public Key Policies` → `Trusted Root Certification Authorities`
   - Right-click → **Import** → Import `RootCA.crt` .

Step 6: Verification & Troubleshooting Protocol

Execute the following commands from an elevated command prompt on each of the 9 target Windows systems (including HQ/CP Server Core DCs):

DOS

```
:: Force GPO update and trigger certificate auto-enrollment pulse
gpupdate /force
certutil -pulse

:: Verify local personal certificate store
certutil -store My
```

Verification Matrix

Target Host	Host Type	GPO Applied	Issued Templa
hq-dc01	Server Core DC	[ ]	POD-DomainContrc
hq-dc02	Server Core DC	[ ]	POD-DomainContrc
cp-dc03	Server Core DC	[ ]	POD-DomainContrc
cp-dc04	Server Core DC	[ ]	POD-DomainContrc
hq-ca01	Subordinate CA	[ ]	POD-Compute
APC-01	Member Server	[ ]	POD-Compute
APC-02	Member Server	[ ]	POD-Compute
APC-03	Member Server	[ ]	POD-Compute

HQ-SRV *(Win)*	Member Server	[ ]	POD-Compute

Verification Test for Revocation and Fetching

Export one issued certificate to a file (e.g., `issued-computer.cer`) and test network-based certificate retrieval and revocation checking from both HQ and CP locations:

DOS

```
certutil -verify -urlfetch issued-computer.cer
```

Ensure all HTTP fetches to `[http://pki.local/pki/](http://pki.local/pki/)...` report `Leaf certificate revocation check passed .`